



强化学习：决策智能

Reinforcement Learning: Decision intelligence



授课对象：计算机科学与技术专业 二年级

课程名称：人工智能（专业必修）

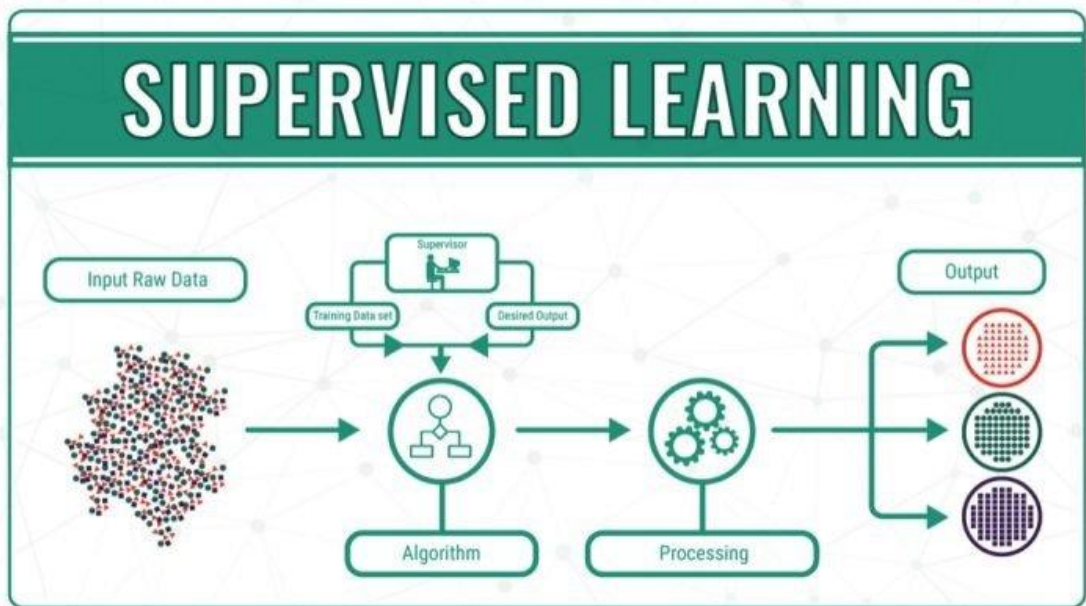
课程学分：3学分



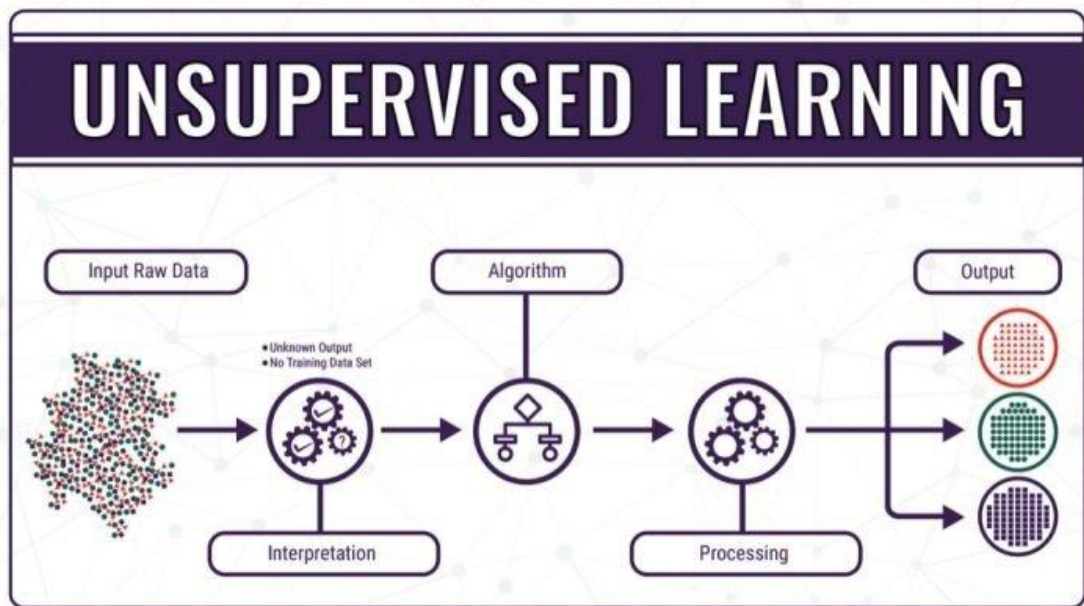


前情知识回顾：机器学习

监督学习：在知道输入和输出的情况下训练出一个模型，被用于寻找输入和输出之间的映射关系。



无监督学习：在训练数据中没有标签或目标值的情况下训练一个模型，被用于探索数据中隐含的模式和分布。

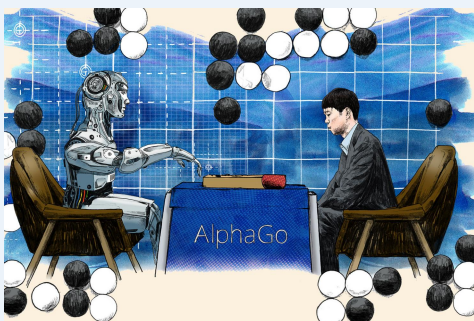


需要提前给定大量静态数据，学习目标是给定数据中的潜在结构或映射



动态决策场景

在大量**序列决策**场景中，智能体需要与环境进行**动态交互**，通过交互获得的数据来学习决策策略。



博弈游戏



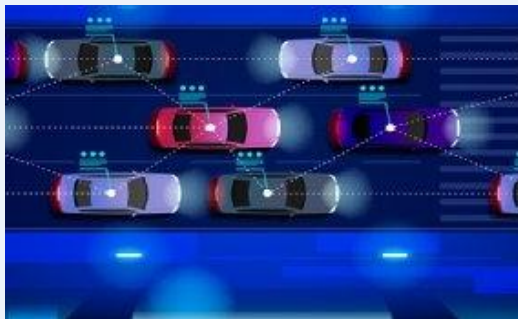
无人机空战



机器人控制



交通灯控制



无人驾驶



智能电网



什么是强化学习?

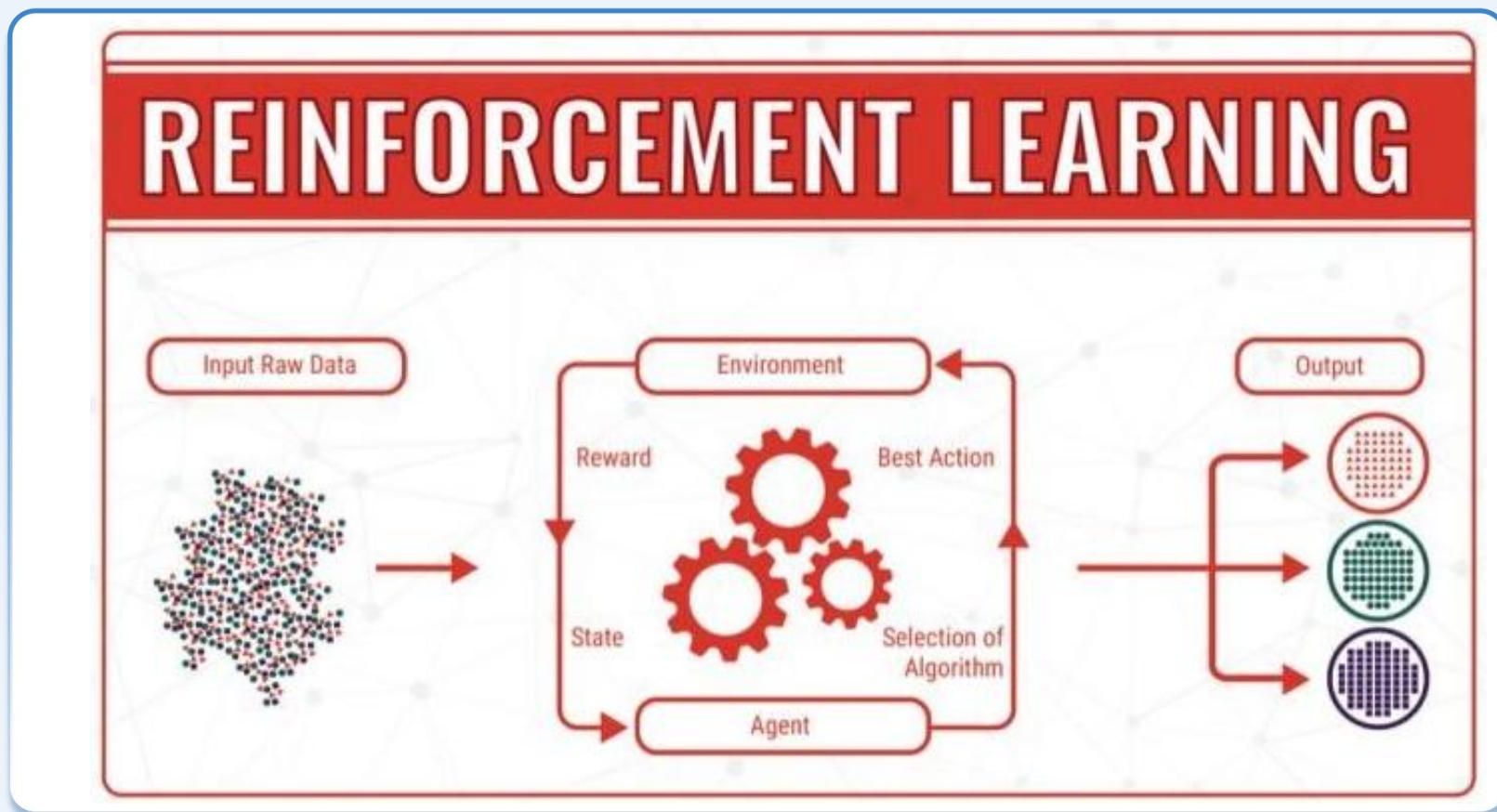
*“Reinforcement learning problems involve learning what to do --- **how to map situations to actions** --- so as to **maximize a numerical reward signal**. In an essential way these are closed-loop problems because the learning system’s actions influence its later inputs. Moreover, the learner is **not told which actions to take**, as in many forms of machine learning, but instead must discover which actions yield the most reward by **trying them out**. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. **These three characteristics** --- **being closed-loop** in an essential way, **not having direct instructions** as to what actions to take, and where the consequences of actions, including reward signals, play out over **extended time periods** --- are the three most important distinguishing features of the reinforcement learning problem”*

——Richard S. Sutton



什么是强化学习?

强化学习讨论的问题是：在一个复杂不确定的环境中，智能体如何通过与环境进行大量试错来学习一个最优决策策略。





什么是强化学习?

相比于监督学习和无监督学习，强化学习是机器学习的第三种学习范式

监督学习：

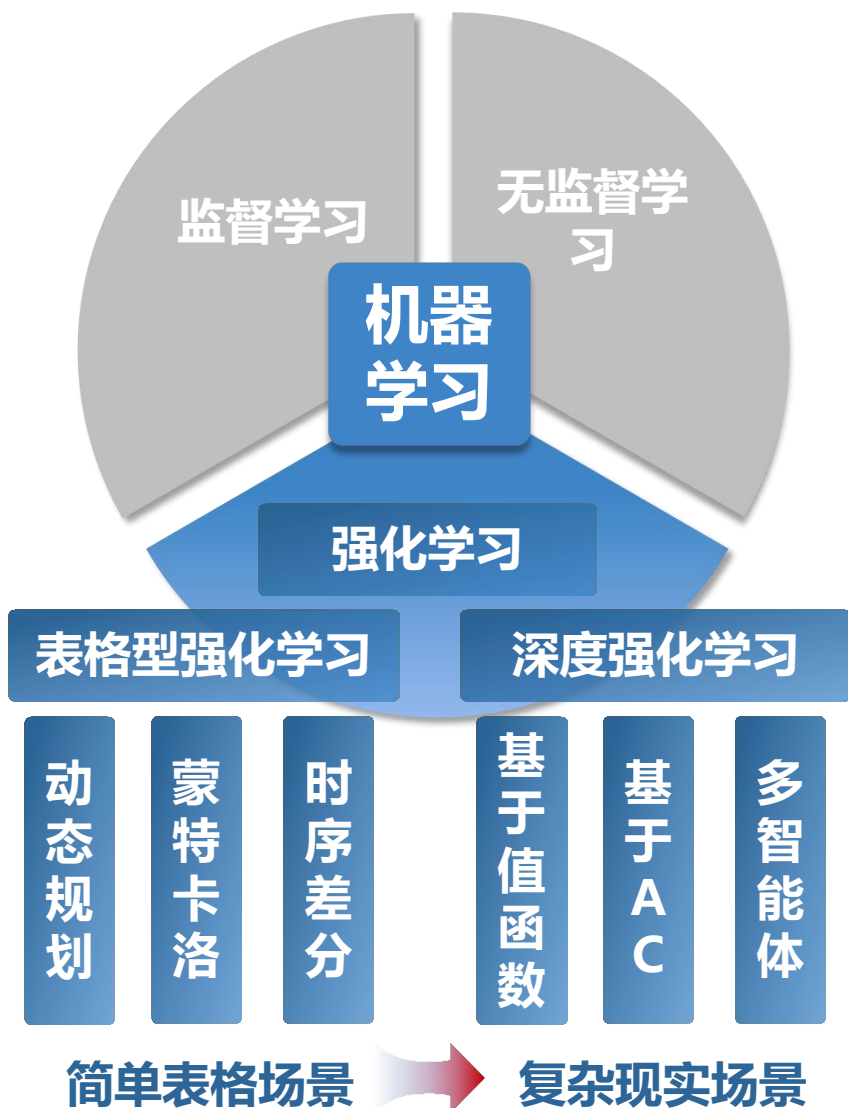
- 分类或预测标签
- 没有交互
- 没有序列决策
- 没有探索（试错）

无监督学习：

- 发现数据的内部模式或结构
- 没有交互
- 没有序列决策
- 没有探索（试错）

强化学习：

- **试错学习**：通过尝试不同的动作，并根据收到的奖励信号调整策略来学习最优行为。
- **延迟奖励**：可能需要经过一系列动作后才能得到最终的奖励，因此必须考虑长期回报。
- **动态策略**：智能体需要根据当前状态选择行动，选择动作的策略是动态调整的。
- **序列决策**：智能体和环境的交互是一个序列决策过程。



本章节知识脉络

学习目标

重点 / Importance

1. **理解**强化学习基本概念
2. **掌握**表格型强化学习三类算法
3. **掌握**深度强化学习基本算法
4. **了解**多智能体强化学习相关概念和算法

难点 / Difficulty

1. 建模强化学习问题的形式化过程
2. 训练强化学习算法在复杂场景上的策略模型

目标 / Goal

能**灵活应用**强化学习算法到任意复杂环境

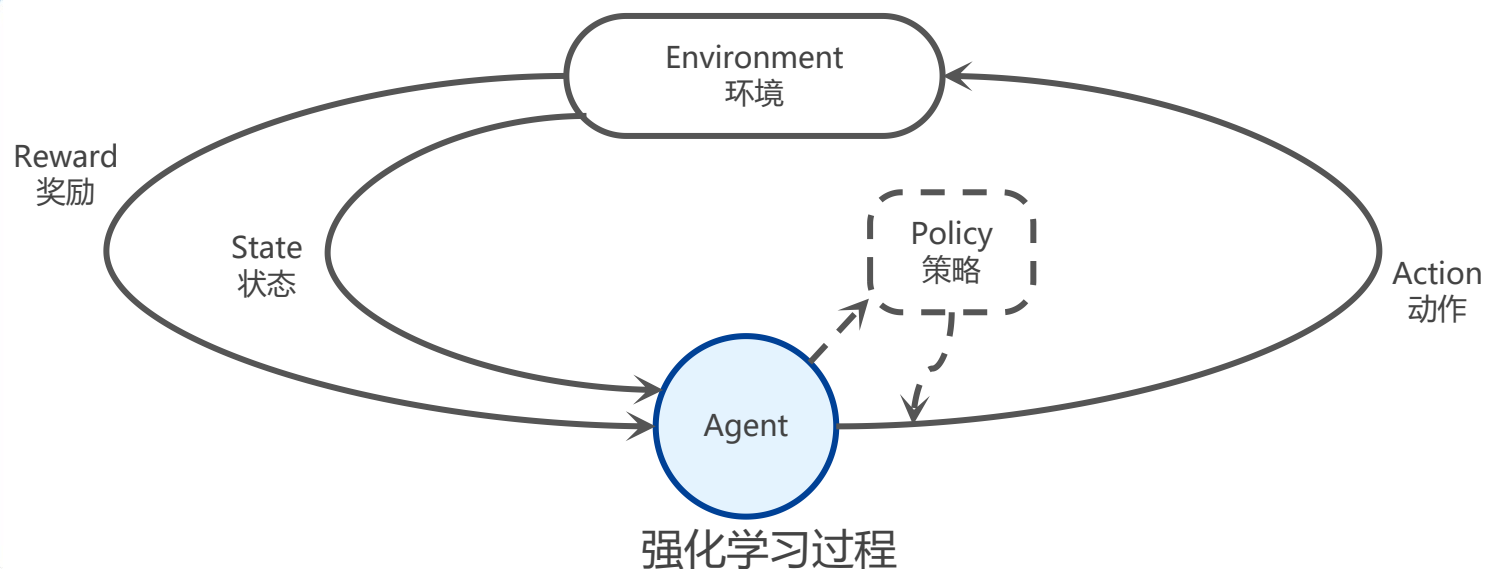
智能体和环境

强化学习 (Reinforcement Learning, RL)

- 受动物的学习过程启发，在试错中学习
- 不断与环境交互，获取学习所需的经验

四个基本要素

1. 环境模型
2. 交互样本 (状态、动作)
3. 策略
4. 奖励



智能体和环境

- 智能体 (Agent) : 游戏中玩家所操控的角色
- 环境 (Environment) : 道路、障碍物、怪物等智能体外的其它元素



《冒险岛》游戏中的状态与动作

状态

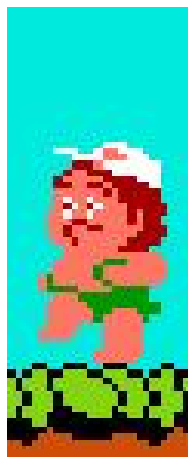
- 状态 (State) : 当前帧的画面



《冒险岛》游戏中的状态与动作

动作

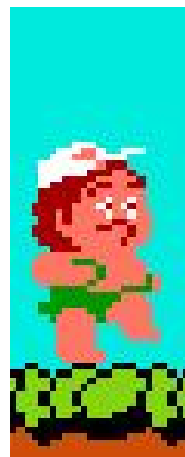
- 动作 (action) : 当前可以采取的行动



向后走



向上跳



向前走



《冒险岛》游戏中的状态与动作

奖励

- 奖励 (reward) : 一般对应着相关任务需求



《冒险岛》游戏中存在的奖励

正向奖励:

- 收集苹果: +50
- 攻击小怪: +100
- ...

负向奖励:

- 每消耗一秒时间: -5
- 接触小怪: $-\infty$ (游戏结束)
- ...

强化学习的目标就是最大化在环境中所能获得的奖励。

马尔科夫决策过程

- **马尔可夫性质**：给定当前状态后，未来状态与过去状态（即该过程的历史状态）是条件独立的。

$$Pr\{S_{t+h} = s | S_i = s_i, i \leq t\} = Pr\{S_{t+h} = s | S_t = s_t\}, \forall h > 0$$

- 当前状态涵盖了历史中的所有相关信息
- 一旦当前状态已知即可忽略其余历史状态信息



历史状态



当前状态

《冒险岛》游戏中的马尔可夫性质

马尔科夫决策过程

- 马尔可夫决策过程 (Markov Decision Process, MDP) : 对于一个马尔可夫决策过程, 可以用一个五元组 $\langle \mathcal{S}, \mathcal{A}, p, R, \gamma \rangle$ 来表示:
 - 状态空间 $\mathcal{S}$ 是所有状态组成的集合
 - 动作空间 $\mathcal{A}$ 是所有动作组成的集合
 - 转移函数 p 刻画了状态之间的转移概率:
$$p(s'|s, a) = \Pr\{S_{t+1} = s' | S_t = s, A_t = a\}$$
 - R 表示奖励函数, 不止取决于当前的状态, 还受到动作的影响:
$$R_t = R(S_t, A_t, S_{t+1})$$
 - γ 表示折扣因子, $\gamma \in [0, 1]$

策略函数

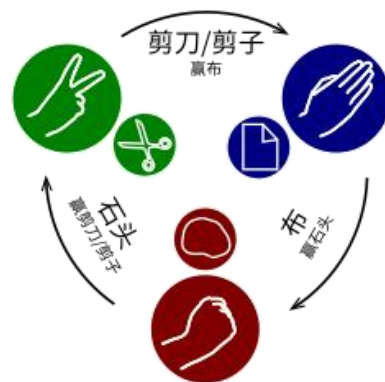
- 策略 (policy) : 从状态空间到动作空间的映射
- 随机性策略 $\pi(a|s)$: 输出在状态 s 下选择动作 a 的概率
- 确定性策略 $\pi(s)$: 输出在状态 s 下选择的确定动作

$$\sum_{a \in \mathcal{A}} \pi(a|s) = 1$$

$$a = \pi(s)$$



确定性策略可以在《冒险岛》中取得不错的表现

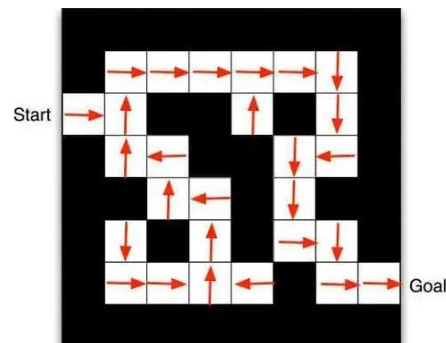
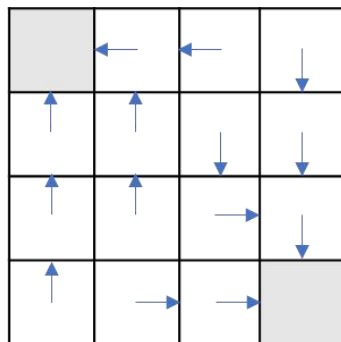


石头剪刀布中则更适合使用随机性策略

策略函数

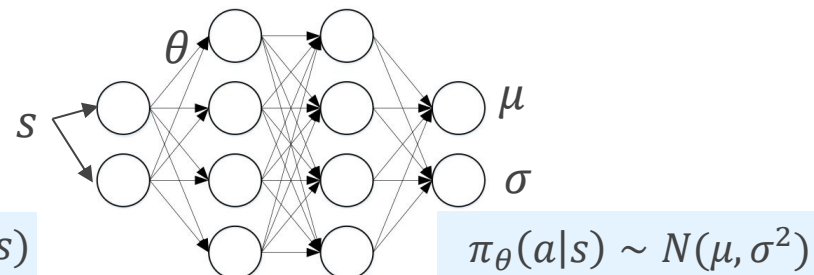
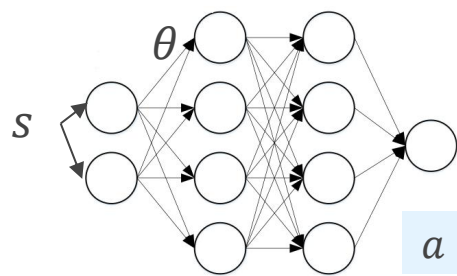
策略的具体形式

- 表格策略



- 参数化策略

$$\pi_{\theta}(s) \triangleq \pi(s; \theta)$$
$$\pi_{\theta}(a|s) \triangleq \pi(a|s; \theta)$$

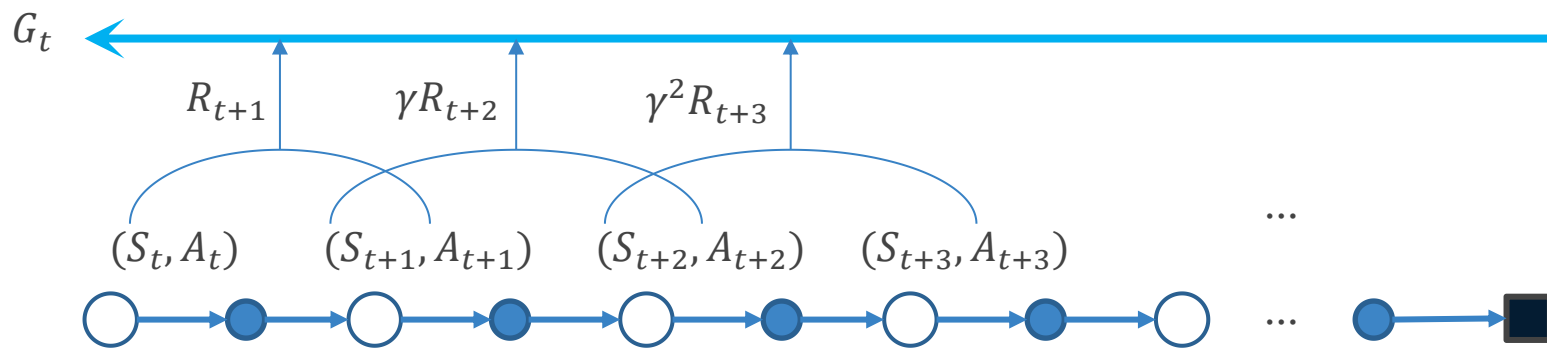


回报

- **回报 (return)** : 回报 G_t 为从时间步 t 开始的折扣奖励总和

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- 折扣因子 $\gamma \in [0,1]$ 用于衡量未来奖励在当前所具备的价值
- $k+1$ 步后的奖励 R 在当前的价值被定义为 $\gamma^k R$

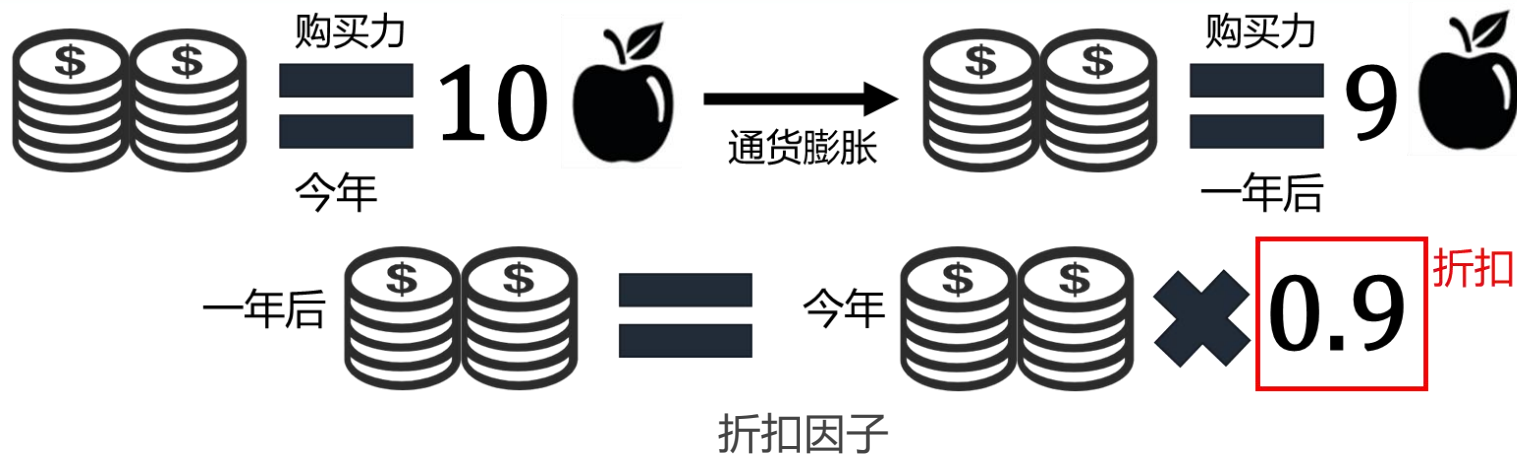


强化学习着眼于**最大化累积奖励**，也即上述定义的回报 (return)。

回报

折扣因子的意义：

- 加入折扣后可以获得良好的数学性质，如确保一些算法的收敛性
- 避免马尔可夫过程中的循环导致的无限奖励
- 未来的奖励具有不确定性
- 动物和人类在实际场景中也更倾向于即时奖励
- 考虑真实的经济情况，当前收入相比未来收入具有更强的购买力



价值函数

价值函数用于评估一个策略的好坏

- 状态价值函数 (State-value function) : 以状态 s 为起始状态且按照策略 π 行动的期望回报。

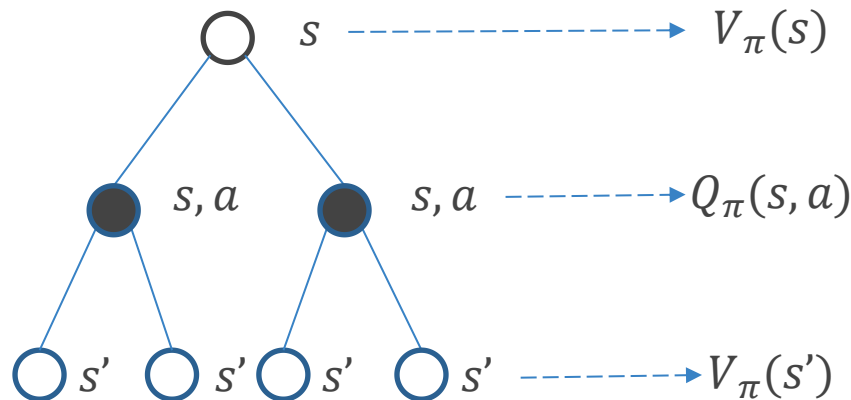
$$V_{\pi}(s) \triangleq \mathbb{E}_{\pi}[G_t | S_t = s]$$

- 动作价值函数 (Action-value function) : 以状态 s 为起始状态且采取动作 a , 然后按照策略 π 行动得到的回报。

$$Q_{\pi}(s, a) \triangleq \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

$$\sum_{a \in \mathcal{A}} \pi(a|s)$$

$$\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a$$



$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q_{\pi}(s, a)$$

$$Q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_{\pi}(s')$$

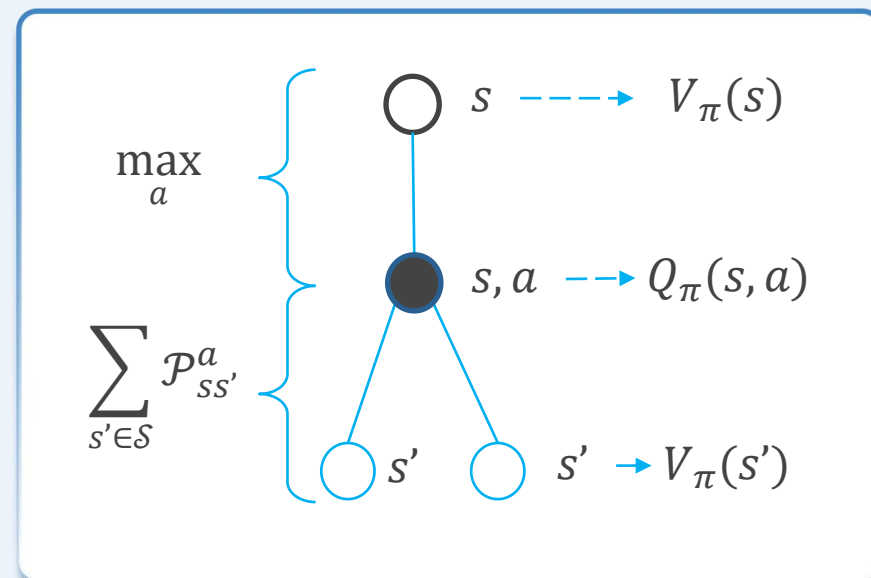
价值函数

- 最优价值函数:

$$V_*(s) = \max_{\pi} V_{\pi}(s), \forall s \in \mathcal{S}$$
$$Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

- $V_*(s)$ 与 $Q_*(s, a)$ 之间的关系

$$V_*(s) = \max_a Q_*(s, a)$$
$$Q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_*(s')$$



最优策略：可以通过 $q_*(s, a)$ 得到一个确定性的最优策略

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in \mathcal{A}} Q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

贝尔曼方程

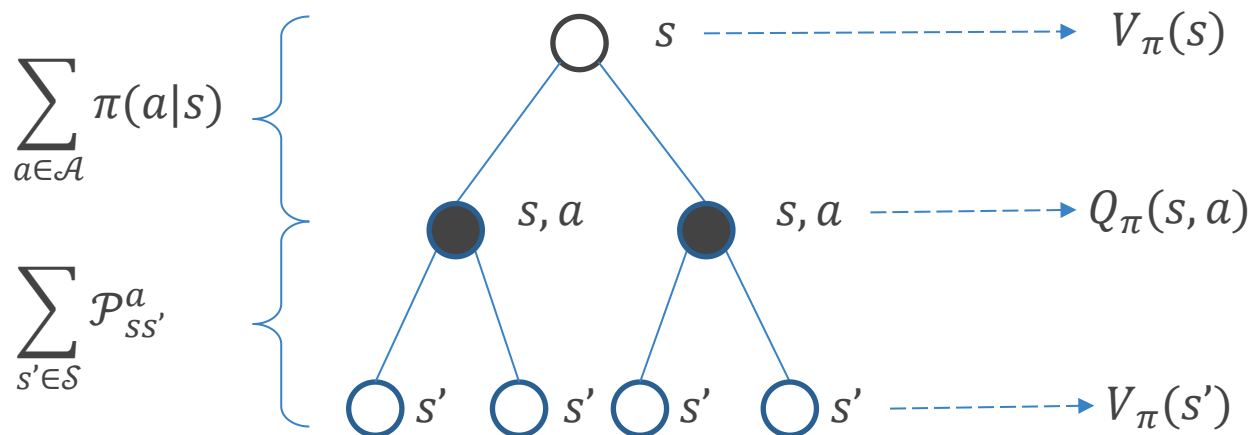
状态价值函数可以被拆分为即时奖励与后续状态的折扣价值

$$\begin{aligned} V_{\pi}(s) &= \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s] \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_{\pi}(s') \right) \end{aligned}$$

贝尔曼方程

动作状态价值函数也可以被拆分为即时奖励与后续状态的折扣价值

$$\begin{aligned} Q_{\pi}(s, a) &= \mathbb{E}_{\pi}[R_{t+1} + \gamma Q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \\ &= \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_{\pi}(s', a') \end{aligned}$$



贝尔曼最优方程

- 最优状态价值函数:

$$\begin{aligned} V_*(s) &= \max_{\pi} V_{\pi}(s) \\ &= \max_{\pi} \mathbb{E}_{\pi}[\mathcal{R}_s^a + \gamma V_{\pi}(s')] \\ &= \max_{\pi} \mathbb{E}_{\pi}[\mathcal{R}_s^a + \gamma V_*(s')] \\ &= \max_a (\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')) \end{aligned}$$

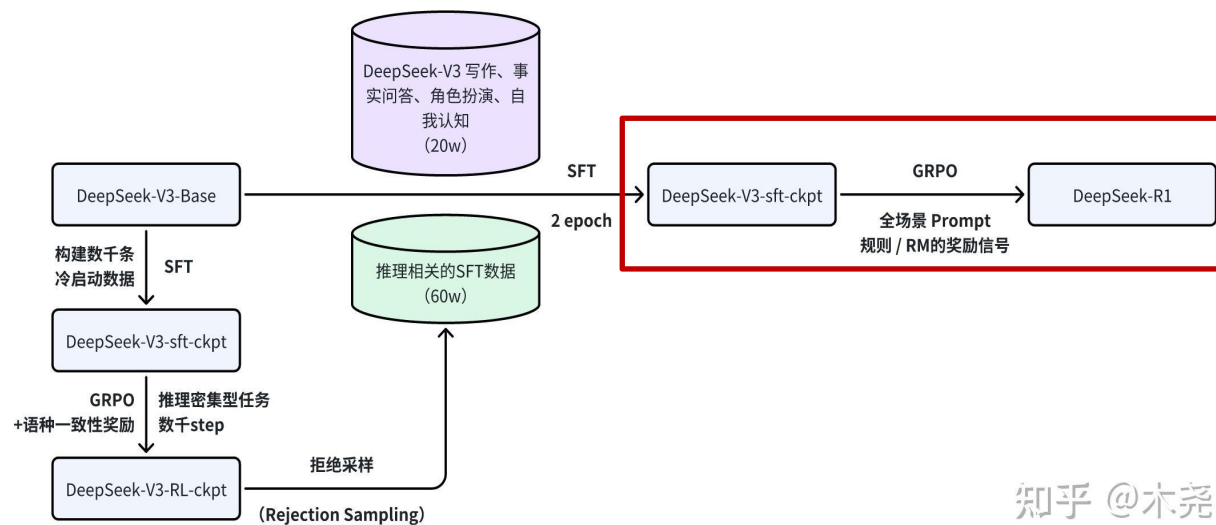
- 最优动作价值函数:

$$\begin{aligned} Q_*(s, a) &= \max_{\pi} Q_{\pi}(s, a) \\ &= \max_{\pi} \mathbb{E}_{\pi}[\mathcal{R}_s^a + \gamma Q_{\pi}(s', a')] \\ &= \mathbb{E}_{\pi} \left[\mathcal{R}_s^a + \gamma \max_{a'} Q_*(s', a') \right] \\ &= \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} Q_*(s', a') \end{aligned}$$

AI for Science

Paper	Publisher	Application
Avoiding fusion plasma tearing instability with deep reinforcement learning	<i>Nature</i> 2024	Tokamak control
Champion-level drone racing using deep reinforcement learning	<i>Nature</i> 2023	Drone racing
Top-down design of protein architectures with reinforcement learning	<i>Science</i> 2023	Protein design
Dense reinforcement learning for safety validation of autonomous vehicles	<i>Nature</i> 2023	Autonomous driving
Magnetic control of tokamak plasmas through deep reinforcement learning	<i>Nature</i> 2022	Tokamak control
Discovering faster matrix multiplication algorithms with reinforcement learning	<i>Nature</i> 2022	Matrix multiplication
A graph placement methodology for fast chip design	<i>Nature</i> 2021	Chip design
A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play	<i>Science</i> 2018	Board game

应用——大模型





应用——机器人

Real-World Humanoid Locomotion with Reinforcement Learning

Ilija Radosavovic*, Tete Xiao*, Bike Zhang*
Trevor Darrell†, Jitendra Malik†, Koushil Sreenath†

University of California, Berkeley



应用——机器人

Fully Autonomous Real-World Reinforcement Learning for Mobile Manipulation

In submission, CoRL 2021

应用——无人机

Champion-Level Performance in Drone Racing using Deep Reinforcement Learning

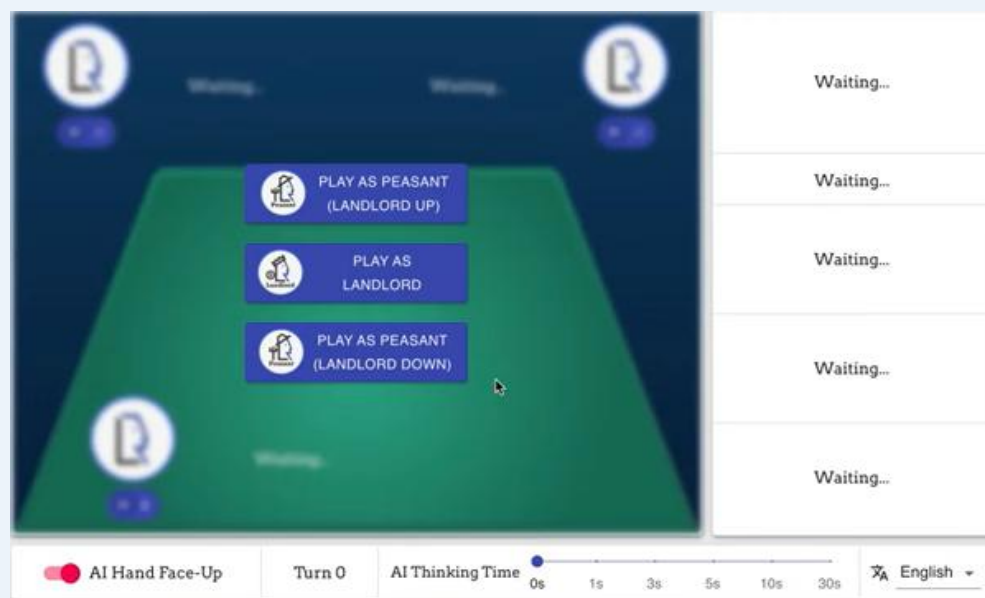
E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza



University of
Zurich^{UZH}



应用——博弈



DouZero



AlphaGo

应用自动驾驶



Swaayatt
Robots

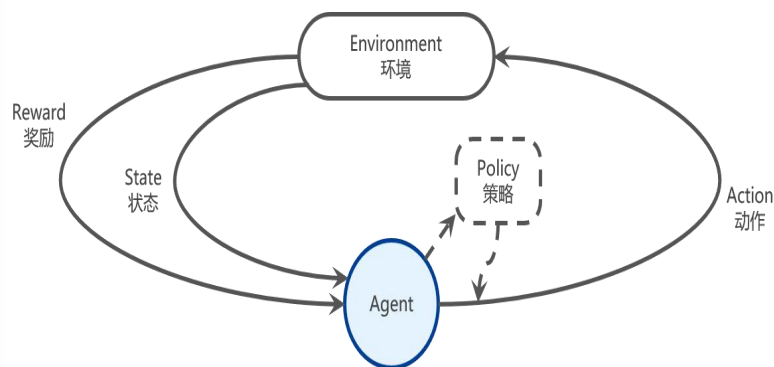
近年来重要应用

Paper	Publisher	Application
Whole-body physics simulation of fruit fly locomotion	<i>Nature</i> 2025	Whole-body model of the fruit fly
Mastering diverse control tasks through world models	<i>Nature</i> 2025	Game play
Experiment-free exoskeleton assistance via learning in simulation	<i>Nature</i> 2024	Exoskeleton assistance
Avoiding fusion plasma tearing instability with deep reinforcement learning	<i>Nature</i> 2024	Tokamak control
Champion-level drone racing using deep reinforcement learning	<i>Nature</i> 2023	Drone racing
Top-down design of protein architectures with reinforcement learning	<i>Science</i> 2023	Protein design
Dense reinforcement learning for safety validation of autonomous vehicles	<i>Nature</i> 2023	Autonomous driving
Magnetic control of tokamak plasmas through deep reinforcement learning	<i>Nature</i> 2022	Tokamak control
Discovering faster matrix multiplication algorithms with reinforcement learning	<i>Nature</i> 2022	Matrix multiplication
A graph placement methodology for fast chip design	<i>Nature</i> 2021	Chip design
A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play	<i>Science</i> 2018	Board game

本节小结

强化学习的基本概念

- 智能体
- 环境
- 动作
- 奖励



马尔可夫决策过程

- 五元组 $\langle \mathcal{S}, \mathcal{A}, p, R, \gamma \rangle$

- 确定性策略函数 $\sum_{a \in \mathcal{A}} \pi(a|s) = 1$

- 随机策略函数 $a = \pi(s)$

- 累计折扣回报

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- 状态值函数 $V_{\pi}(s) \triangleq \mathbb{E}_{\pi}[G_t | S_t = s]$

- 动作值函数

$$Q_{\pi}(s, a) \triangleq \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

贝尔曼方程

- 状态值函数定义

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_{\pi}(s') \right)$$

- 动作值函数定义

$$Q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_{\pi}(s', a')$$

- 最优状态值函数定义

$$V_*(s) = \max_a (\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s'))$$

- 最优动作值函数定义

$$Q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} Q_*(s', a')$$

表格型强化学习场景

- 右图展示了 Gym 的**冰湖环境 (Frozen Lake)**，智能体起点状态在左上角，目标状态在右下角，中间还有若干冰洞。
- 每一个状态都可以采取上、下、左、右 4 个动作。由于智能体在冰面行走，因此每次行走不一定会朝着预想方向，而是有一定的概率向与预想方向垂直的方向滑行。例如本来想向前走，最后却走到了左边或右边。
- 当到达宝箱位置，智能体获得正奖励值，否则没有奖励值。
- 当掉入冰洞或到达目标状态时结束。



冰湖路径图

表格型强化学习场景

- 冰湖环境较为简单，我们可以很容易地构建所有状态动作对的Q表。

状态 \ 动作	上	下	左	右
s_0	0.0	1.0	0.0	0.0
s_1	0.0	1.0	0.0	1.0
s_2	0.0	0.5	0.0	0.5
s_3	0.5	2.0	0.0	0.5
s_4	0.0	0.0	0.0	0.5
...	...	...	...	...

Q表图

- 根据 Q 表，我们可以知道此时每个状态对应的最优策略，即通过每个动作对应的 Q 值大小判断。对于状态 s_0 ，最优策略是采取动作“下”，状态 s_1 有两个最优动作“下”和“右”。



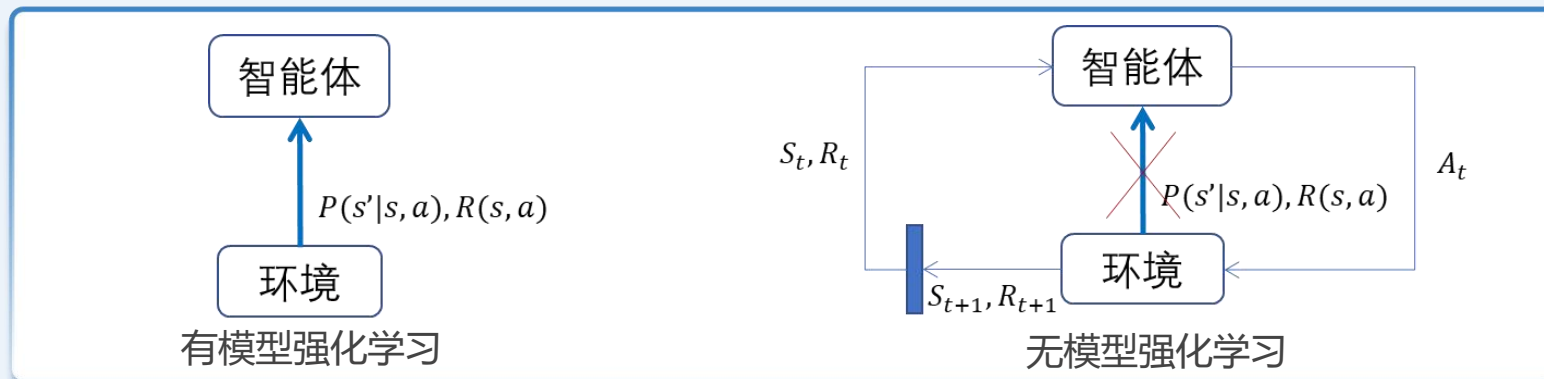
冰湖路径图

表格型强化学习算法

通常，Q 表会被初始化为全 0，通过智能体和环境交互的结果不断更新 Q 表。当 Q 表中的每个值函数更新幅度小于某个阈值时，表示着 Q 表已经收敛，那么每个状态对应的策略就是强化学习问题的最优策略。

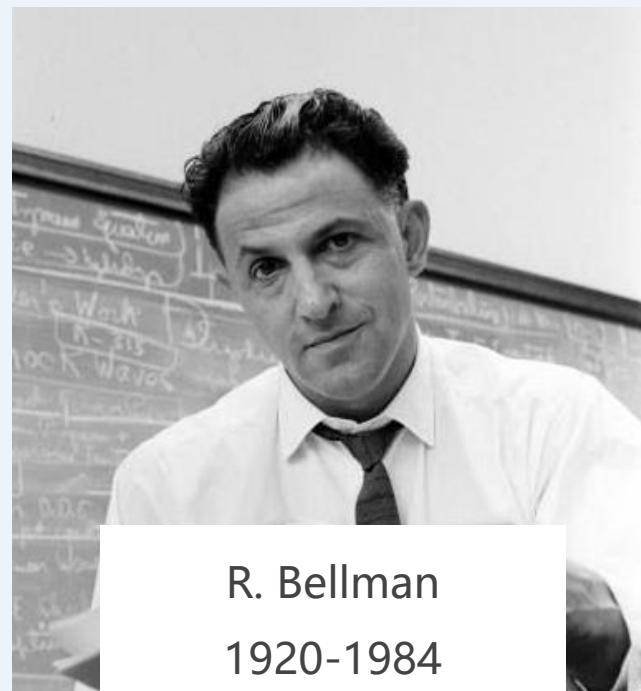
Q 表的更新方法主要分为两大类：

- **有环境模型的求解方法**：需要知道环境的状态转移函数以及奖励函数，通过**动态规划方法**递归求解最优策略，常用求解方法主要包括：策略迭代算法、值迭代算法；
- **无环境模型的求解方法**：无需显式知道环境的状态转移函数和奖励函数，而是通过智能体与环境交互的数据更新 Q 值，常用求解方法主要包括：**蒙特卡洛方法**，**时序差分方法**



动态规划算法

- 动态规划 (Dynamic Programming, DP) 是一种适用于解决具有**最优子结构 (Optimal Substructure)** 和**重叠子问题 (overlapping subproblem)** 两个性质的算法思想。
- 20 世纪 50 年代初, 美国数学家**贝尔曼 (R.Bellman)** 等人在研究多阶段决策过程的优化问题时, 提出了著名的**最优化原理**, 从而创立了动态规划。
- 该方法应用十分广泛, 包括但不限于**工程技术、经济、工业生产、军事**等。



R. Bellman

1920-1984

动态规划算法

性质一

最优子结构

- 原问题可以拆分成若干个子问题
- 全局最优解可以拆分成子问题的最优解



贝尔曼方程含有递归分解

性质二

重叠子问题

- 子问题可能会出现多次
- 子问题的解可以被记录并重复利用



价值函数存储了状态价值并被重复利用

策略迭代

价值迭代

马尔可夫决策过程

动态规划算法

策略评估 (Policy Evaluation) : 评估给定的策略 π
(求解该策略对应的价值函数)

- 迭代调用贝尔曼期望方程进行更新
- 使用同步更新 (每次迭代更新所有状态的价值)

对于 $k = 1:H$ (H 是让价值函数收敛所需的迭代次数) 对于所有状态 $s \in \mathcal{S}$

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

- 右侧简要给出了该方法的收敛性证明

收敛性证明:

利用 ∞ 范数度量两个状态价值函数 u 与 v 之间的距离, 即状态价值的最大差异:

$$\|u - v\|_{\infty} = \max_{s \in \mathcal{S}} |u(s) - v(s)|$$

定义贝尔曼期望算子 B_{π} :

$$B_{\pi} v_k = R_{\pi} + \gamma P_{\pi} v_k$$

该算子是一个 γ -压缩:

$$\begin{aligned} \|B_{\pi} u - B_{\pi} v\|_{\infty} &= \|(R_{\pi} + \gamma P_{\pi} u) - (R_{\pi} + \gamma P_{\pi} v)\|_{\infty} \\ &= \|\gamma P_{\pi} (u - v)\|_{\infty} \leq \|\gamma P_{\pi}\| \|u - v\|_{\infty} \\ &\leq \gamma \|u - v\|_{\infty} \end{aligned}$$

由压缩映射定理可知 v_{π} 是 B_{π} 的唯一不动点, 因此策略评估能够收敛到 v_{π}

动态规划算法

策略评估 (Policy Evaluation) : 评估给定的策略 π
(求解该策略对应的价值函数)

- 迭代调用贝尔曼期望方程进行更新
- 使用同步更新 (每次迭代更新所有状态的价值)

对于 $k = 1:H$ (H 是让价值函数收敛所需的迭代次数) 对于所有状态 $s \in \mathcal{S}$

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

- 右侧简要给出了该方法的收敛性证明

Algorithm 1.1 策略评估

Input: 策略 $\pi(s)$, 状态转移 $P(s'|s, a)$, 奖励函数 $R(s, a)$, 阈值 ϵ , 衰减因子 γ , 值函数 $V(s)$

Output: 值函数 $V_\pi(s)$

```
1:  $k \leftarrow 0$ ,  $V_k(s) \leftarrow V(s)$ 。  
2: for  $k = 0, 1, 2, \dots$  do  
3:   for 每个状态  $s \in \mathcal{S}$  do  
4:     利用公式 1.25 计算  $V_{k+1}(s)$   
5:   end for  
6:   if  $|V_{k+1} - V_k| < \epsilon$  then  
7:      $V_\pi \leftarrow V_{k+1}$   
8:     return  $V_\pi$   
9:   end if  
10: end for
```

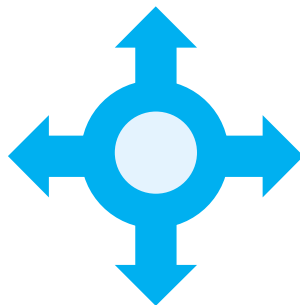
动态规划算法

网格环境中的策略评估

环境

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

动作



每次移动会有 -1 的奖励

奖励

	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	

- 不考虑折扣 ($\gamma = 1$)
- 左上角与右下角为终止状态，其余为非终止状态
- 到达终止状态之前，每次移动获得 -1 的奖励
- 智能体采用一个随机策略

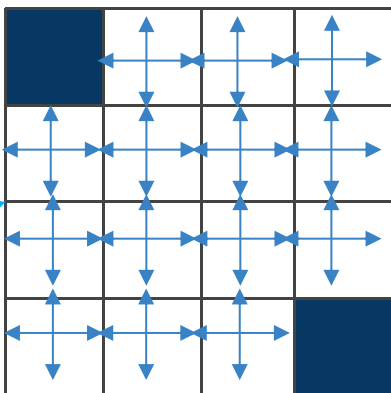
动态规划算法

网格环境中的策略评估

随机策略的状态价值
 $v_k(s)$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

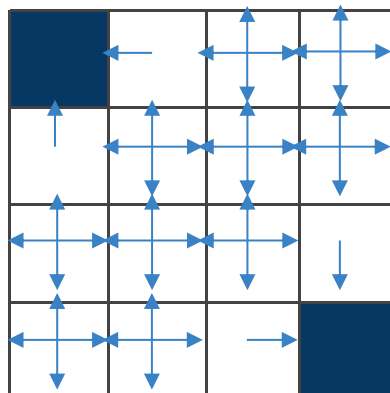
k=0



随机策略

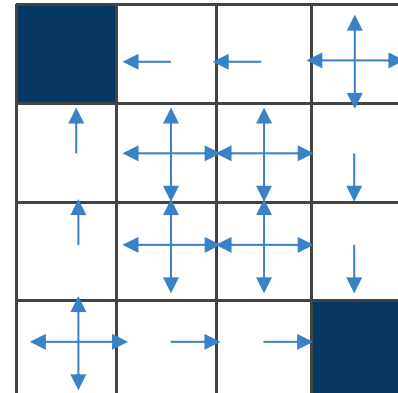
0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

k=1



0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

k=2



依据 $v_k(s)$ 的贪心策略

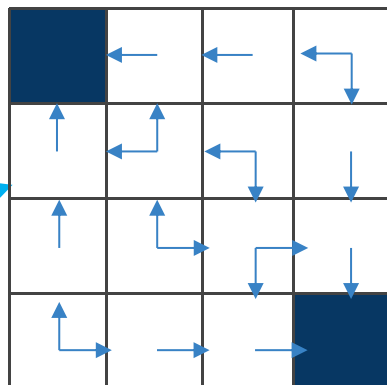
动态规划算法

网格环境中的策略评估

随机策略的状态价值
 $v_k(s)$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

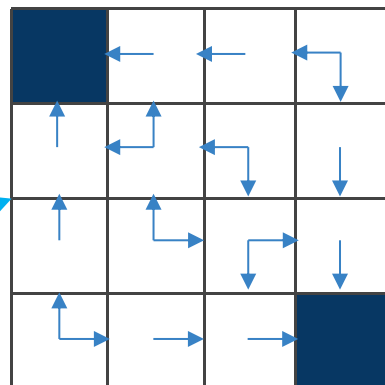
k=3



最优策略

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

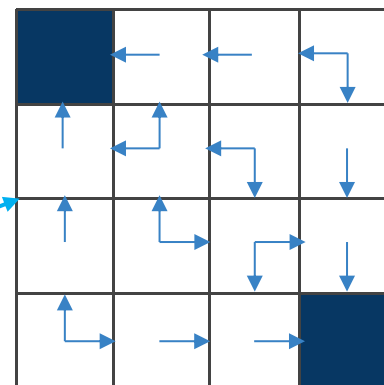
k=10



最优策略

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

k趋近 ∞



最优策略

动态规划算法

策略提升 (Policy Improvement) : 依据策略评估得到的值函数, 改进策略 π

- 根据价值函数得到一个更优的策略
- 可以依据价值函数定义策略之间的偏序关系

$$\pi \geq \pi' \text{ if } v_{\pi}(s) \geq v_{\pi'}(s), \forall s \in \mathcal{S}$$

- 对于一个**确定性策略** ($a = \pi(s)$) , 可以通过贪心的**最大化动作价值函数**得到一个新策略

$$\pi'(a|s) = \operatorname{argmax}_{a \in \mathcal{A}} q_{\pi}(s, a)$$

而策略 π' 比策略 π 更好或至少一样好



高手玩家 $\rightarrow v_{\pi_{\text{高手}}}(s)$ 较大

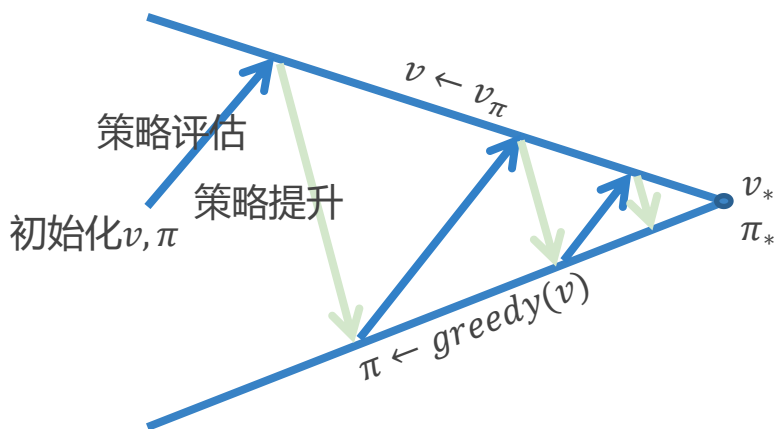
新手玩家 $\rightarrow v_{\pi_{\text{新手}}}(s)$ 较小

$$\begin{aligned} q_{\pi}(s, \pi'(s)) &= \max_{a \in \mathcal{A}} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s) \\ v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) | S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, \pi'(S_{t+2})) | S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s] = v_{\pi'}(s) \end{aligned}$$

动态规划算法

策略迭代 (Policy Iteration)

- 策略评估 估计原策略价值函数 v_π
 - ✓ 迭代策略评估
- 策略提升 生成新策略 $\pi' \geq \pi$
 - ✓ 贪心策略提升



Algorithm 1.2 策略迭代

Input: 策略 $\pi(s)$, 状态转移 $P(s'|s, a)$, 奖励函数 $R(s, a)$, 阈值 ϵ , 衰减因子 γ , 值函数 $V(s)$

Output: 最优策略 $\pi_*(s)$, 最优值函数 $V_{\pi_*}(s)$

- 1: $k \leftarrow 0$, $\pi_k(s) \leftarrow \pi(s)$, $V_k(s) \leftarrow V(s)$ 。
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: 利用策略评估得到当前策略 $\pi_k(s)$ 的值函数 $V_k(s)$
- 4: 利用策略改进公式 1.26 得到新策略 $\pi_{k+1}(s)$
- 5: **if** $\pi_{k+1}(s) = \pi_k(s)$ **then**
- 6: $\pi_*(s) \leftarrow \pi_k(s)$, $V_{\pi_*}(s) \leftarrow V_k(s)$
- 7: **return** $\pi_*(s)$, $V_{\pi_*}(s)$
- 8: **end if**
- 9: **end for**

动态规划算法

值迭代 (Value Iteration)

任意的最优策略都可以分解为两个部分

- 最优的第一个动作 A_*
- 对于后继状态 S' 的最优策略 π_*

定理

策略 $\pi(a|s)$ 能够达到状态 s 的最优价值, $v_\pi(s) = v_*(s)$, 当且仅当对于状态 s 的任意后继状态 s' , 策略 π 均能达到其最优价值, $v_\pi(s') = v_*(s')$

如果知道子问题的解 $v_*(s')$, 则 $v_*(s)$ 可以通过下式得到:

$$v_*(s) \leftarrow \max_a \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right)$$

价值迭代的思路即是迭代应用上式直至收敛

动态规划算法

值迭代 (Value Iteration)

- 迭代的调用贝尔曼最优方程进行更新
- 使用同步更新（每次迭代更新所有状态的价值）

对于 $k = 1:H$ (H 是迭代次数)

对于所有状态 $s \in \mathcal{S}$

$$v_{k+1}(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s')$$

不同于策略迭代，价值迭代中没有显式的策略

Algorithm 1.3 值迭代

Input: 状态转移 $P(s'|s, a)$, 奖励函数 $R(s, a)$, 阈值 ϵ , 衰减因子 γ , 值函数 $V(s)$

Output: 最优策略 $\pi_*(s)$, 最优值函数 $V_{\pi_*}(s)$

```
1:  $k \leftarrow 0, V_k(s) \leftarrow V(s)$ 。  
2: for  $k = 0, 1, 2, \dots$  do  
3:   for 每个状态  $s \in S$  do  
4:     利用公式 1.30 计算  $V_{k+1}(s)$   
5:   end for  
6:   if  $|V_{k+1} - V_k| < \epsilon$  then  
7:      $V_{\pi_*}(s) \leftarrow V_{k+1}$   
8:     利用公式 1.31 计算最优策略  $\pi_*(s)$   
9:     return  $\pi_*(s), V_{\pi_*}(s)$   
10:  end if  
11: end for
```

动态规划算法

网格环境中的值迭代

g			

问题

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

 v_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

 v_2

g	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

 v_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

 v_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

 v_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

 v_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

 v_7

动态规划算法

求解问题	贝尔曼方程	算法
预测（求解价值函数）	贝尔曼期望方程	策略评估
控制（求解最优策略）	贝尔曼期望方程+ 贪心策略提升	策略迭代
控制（求解最优策略）	贝尔曼最优方程	价值迭代

- 以上算法都基于状态价值函数 $v_*(s)$ 或 $v_\pi(s)$ ，对于 n 个状态、 m 个动作的MDP，每次迭代的复杂度为 $O(mn^2)$
- 也可以应用于动作价值函数 $q_*(s)$ 或 $q_\pi(s)$ ，每次迭代复杂度为 $O(m^2n^2)$

蒙特卡洛估计方法

蒙特卡洛估计方法

- 20世纪40年代，在科学家冯·诺伊曼、**斯塔尼斯拉夫·乌拉姆**和尼古拉斯·梅特罗波利斯于洛斯阿拉莫斯国家实验室为核武器计划工作时，发明了蒙特卡洛估计方法。
- 出于保密需要，该方法需要一个代号，而这个代号由来是因为乌拉姆的叔叔经常在摩纳哥的蒙特卡罗赌场输钱。
- 蒙特卡洛估计方法是一种以概率统计理论为指导的数值计算方法，与之相对的是确定性算法。



蒙特卡洛估计方法

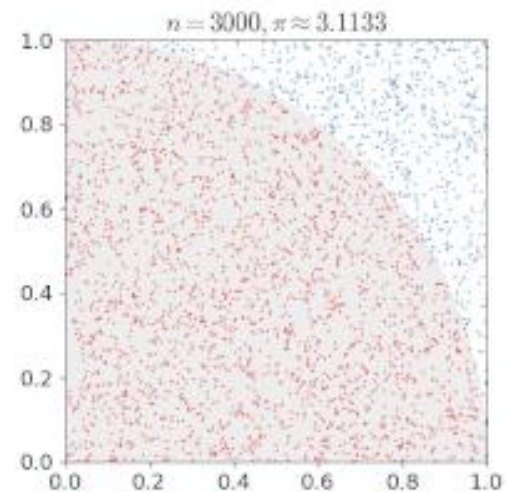
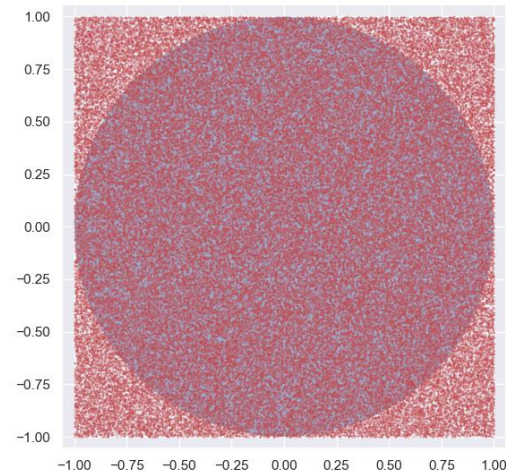
利用蒙特卡洛估计方法估计圆周率

- 设右图中圆的直径=正方形边长= a
- 在正方形内随机生成大量均匀分布的点，易知下式成立

$$\frac{\text{圆内点的个数}}{\text{总的点个数}} \approx \frac{\text{圆形面积}}{\text{正方形面积}} = \frac{\pi \left(\frac{1}{2}a\right)^2}{a^2} = \frac{1}{4}\pi$$

$$\pi \approx 4 * \frac{\text{圆内点的个数}}{\text{总的点个数}}$$

随着采样的数目越多，估计的数值就越精确



蒙特卡洛估计方法

蒙特卡洛方法中的策略评估

- 记 $G^{(i)(j)}(s)$ 为在第 i 个回合中第 j 次访问状态 s 对应的累积回报
- 估计**状态价值函数**:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] \approx \frac{1}{N} \sum_{i=1}^N G^{(i)(j)}(s)$$

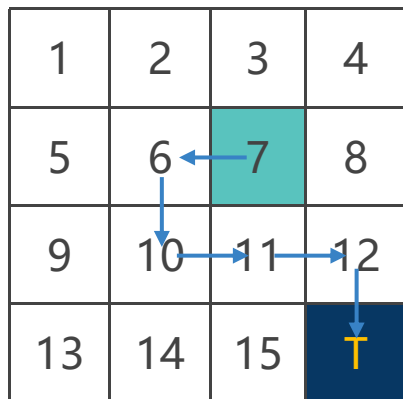
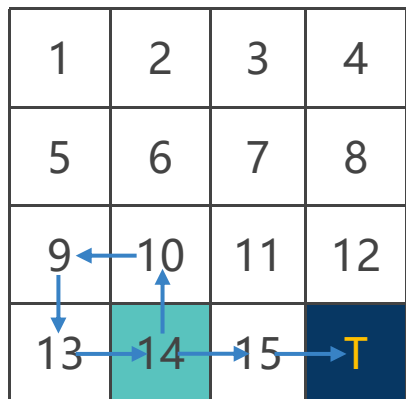
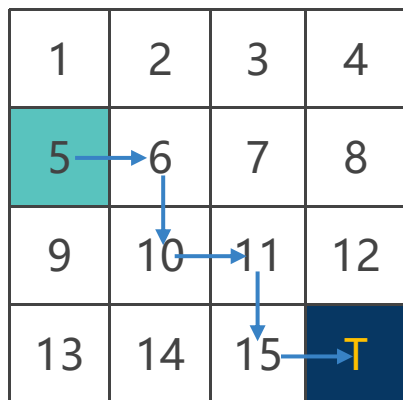
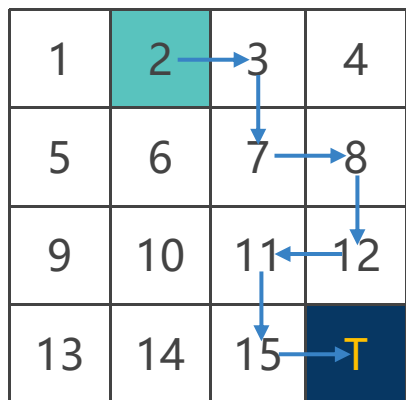
- 记 $G^{(i)(j)}(s, a)$ 为在第 i 个回合中第 j 次访问状态动作对 (s, a) 对应的累积回报
- 估计**动作状态价值函数**:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N G^{(i)(j)}(s, a)$$

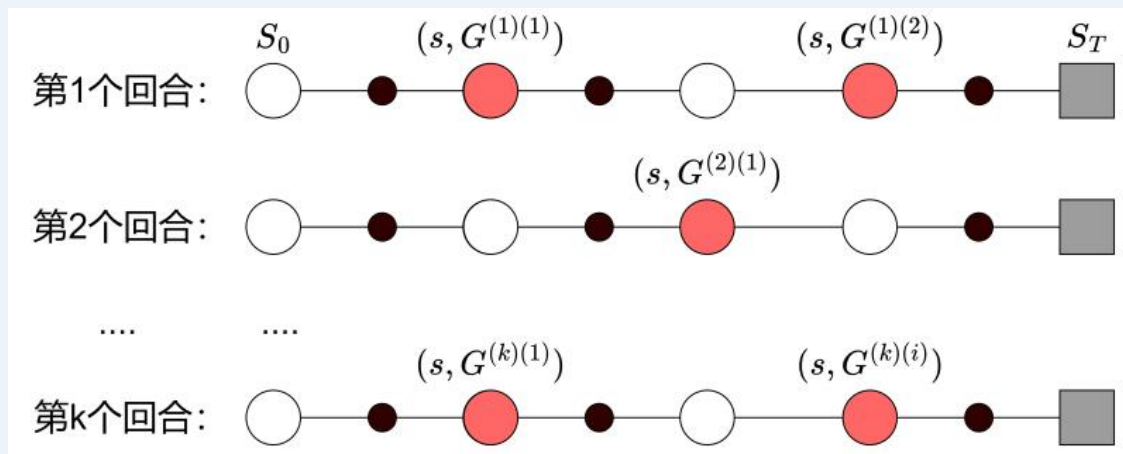
可以发现，蒙特卡洛策略评估
只需要采样数据，而不需要环
境模型！

蒙特卡洛估计方法

蒙特卡洛方法中的策略评估



- 通过采样多条样本轨迹，利用其累积回报的均值来估计状态价值函数



$G^{(i)(j)}(s)$ 表示在第 i 个回合中第 j 次访问状态 s 的回报

蒙特卡洛估计方法

首次访问蒙特卡洛 (First-Visit Monte-Carlo)

- 考虑每个回合中对状态 s 的**第一次**访问:

$$V_N(s) = \frac{G^{(1)(1)}(s) + G^{(2)(1)}(s) + \dots + G^{(N)(1)}(s)}{N}$$

无偏估计量

每次访问蒙特卡洛 (Every-Visit Monte-Carlo)

- 考虑每个回合中对状态 s 的**每一次**访问:

$$V_{N \times M}(s) = \frac{G^{(1)(1)}(s, a) + G^{(1)(2)}(s, a) + \dots + G^{(2)(1)}(s, a) + \dots + G^{(N)(1)}(s, a) + \dots + G^{(N)(M)}(s, a)}{N \times M}$$

有偏估计量

当对状态 s 的访问次数趋于无穷时, 两类方法都会收敛到精确值 $v_\pi(s)$

蒙特卡洛估计方法

蒙特卡洛方法中的策略提升

- 已经估计了状态价值函数 $V_N(s)$ ，策略提升部分为：

$$\operatorname{argmax}_{\Pi} R(s, a) + \gamma \sum_{s'} p(s'|s, a) V_N(s')$$

- 已经估计了动作状态价值函数 $Q_N(s, a)$ ，策略提升部分为：

$$\operatorname{argmax}_{\Pi} Q_N(s, a)$$

可以发现，估计动作状态价值函数不需要知道环境模型，计算更为方便（但也需要更多的数据样本）

蒙特卡洛估计方法

- 策略评估: $Q_N(s, a) = \frac{1}{N} \sum_{i=1}^N G^{(i)}(s, a)$
- 策略提升: $\arg\max_{\Pi} Q_N(s, a)$

优点

- 无需环境模型，只需与环境交互采样样本数据即可学习策略
- 评估某个状态的价值函数与其它状态无关，从而可以只评估部分重要的状态
- 由于不使用贝尔曼方程更新，适用于非马尔可夫场景

缺点

- 若采样的回合个数不够多，容易收敛到次优策略
- 由于需要多个回合的回报，通常只适用有限步长的场景

蒙特卡洛估计方法

蒙特卡洛策略评估收敛需要两个很强的假设

- 需要无限多回合产生的数据
- 需要试探性出发假设产生数据
 - ✓ 也即，对于每一对状态动作对 (s, a) 都有非零概率被选为回合的起点

这两个假设保证了每一个状态动作对 (s, a) 都能被访问到无数次，从而蒙特卡洛策略评估可以收敛到精确的动作状态价值函数。

然而，这两个假设在实践中无法实现。为了得到一个可行的算法，我们需要去除这两个假设。

蒙特卡洛估计方法

蒙特卡洛策略评估收敛需要两个很强的假设

有两类方法可以对第一个假设进行放松

- 在每次策略评估时，通过有限多个回合产生的数据，对精确值计算一个具有一定误差的近似值
 - ✓ 需要做一些假设，来分析逼近误差的幅度和出现概率的上下界，并采取足够多的步数来保证这些界足够小
 - ✓ 这类方法虽然可以保证收敛到较好的近似水平，但在实践中仍然需要大量的回合数据来用于计算
- 在每次策略评估时，采用广义策略迭代的思想，不再要求策略改进前就完成策略评估
 - ✓ 例如，在每一回合结束时，就使用观测到的回报进行策略评估，然后在该回合访问到的每一个状态上进行策略的改进

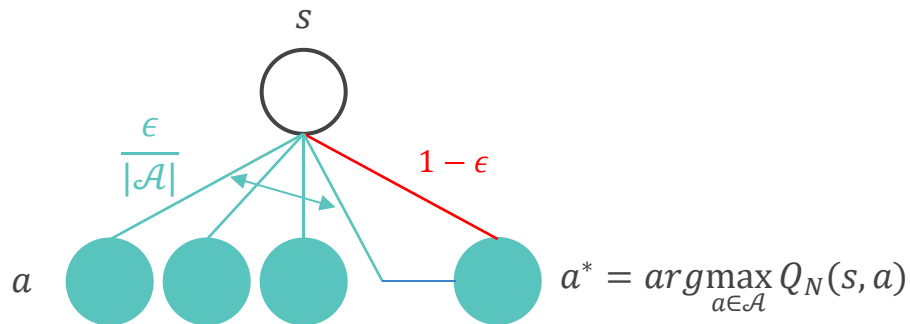
蒙特卡洛估计方法

蒙特卡洛策略评估收敛需要两个很强的假设

通过“软性”策略对第二个假设进行放松

- 一个策略称为**软性**的，若满足任意 s, a , 都有 $\pi(a|s) > 0$
- ϵ -贪心策略是保证充分探索的一个简单软性策略
 - ✓ 以 $1 - \epsilon$ 的概率贪心地选择当前最优动作
 - ✓ 以较小的 ϵ 概率随机选择一个动作

$$\pi(a|s) = \begin{cases} \epsilon/|\mathcal{A}| + 1 - \epsilon & \text{if } a = \arg\max_{a \in \mathcal{A}} Q_N(s, a) \\ \epsilon/|\mathcal{A}| & \text{otherwise} \end{cases}$$



蒙特卡洛估计方法

ϵ -贪心策略满足策略提升定理

- 对于任意策略 π ，依据其动作价值函数 q_π 计算的 ϵ -贪心策略 π' 比原策略 π 好或至少一样好

$$\begin{aligned}
 q_\pi(s, \pi') &= \left(\sum_{a \in \mathcal{A}} \pi'(a|s) q_\pi(s, a) \right) \\
 &= \epsilon |\mathcal{A}| \sum_{a \in \mathcal{A}} q_\pi(s, a) + 1 - \epsilon \left(\max_{a \in \mathcal{A}} q_\pi(s, a) \right) \\
 &\geq \epsilon |\mathcal{A}| \sum_{a \in \mathcal{A}} q_\pi(s, a) + 1 - \epsilon \left(\sum_{a \in \mathcal{A}} \frac{\pi(a|s) (-\epsilon |\mathcal{A}|)}{1 - \epsilon} q_\pi(s, a) \right) \\
 &= \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a) = v_\pi(s)
 \end{aligned}$$

蒙特卡洛估计方法

增量式更新

- 在策略评估部分中，比起存储所有观测到的回报序列并进行均值计算，增量式更新是更高效的计算方法

$$\begin{aligned} Q_N(s, a) &= \frac{1}{N} \sum_{i=1}^N G^{(i)(j)}(s, a) \\ &= \frac{1}{N} [\sum_{i=1}^{N-1} G^{(i)(j)}(s, a) + G^{(N)(1)}(s, a)] \\ &= \frac{N-1}{N} [\frac{1}{N-1} \sum_{i=1}^{N-1} G^{(i)(j)}(s, a)] + \frac{1}{N} G^{(N)(1)}(s, a) \\ &= \frac{N-1}{N} Q_{N-1}(s, a) + \frac{1}{N} G^{(N)(1)}(s, a) \\ &= Q_{N-1}(s, a) + \frac{1}{N} [G^{(N)(1)}(s, a) - Q_{N-1}(s, a)] \end{aligned}$$

蒙特卡洛估计方法

Algorithm 1.4 每次访问蒙特卡洛算法

Input: ϵ , 衰减因子 γ , 最大交互回合数 N

Output: 最优策略 $\pi_*(s)$

- 1: 初始化初始状态 π_0 , 计数器 $Num(s, a) = 0$, 累计回报 $G(s, a) = 0$ 。
- 2: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 3: 利用 π_k 生成一条轨迹 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$
- 4: $g \leftarrow 0$
- 5: **for** $t = T - 1, \dots, 0$ **do**
- 6: 计算状态动作对 (s_t, a_t) 的累计折扣回报 g
- 7: $g \leftarrow r_t + \gamma g$
- 8: $G(s_t, a_t) \leftarrow G(s_t, a_t) + g$
- 9: $Num(s_t, a_t) \leftarrow Num(s_t, a_t) + 1$
- 10: $Q(s_t, a_t) \leftarrow G(s_t, a_t) / Num(s_t, a_t)$
- 11: **end for**
- 12: 利用公式 1.39 更新策略, 得到 π_{k+1}
- 13: **end for**
- 14: **return** $\pi_*(s)$

每次访问蒙特卡洛伪码

Algorithm 1.5 首次访问蒙特卡洛算法

Input: ϵ , 衰减因子 γ , 最大交互回合数 N

Output: 最优策略 $\pi_*(s)$

- 1: 初始化初始状态 π_0 , 计数器 $Num(s, a) = 0$, 累计回报 $G(s, a) = 0$ 。
- 2: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 3: 利用 π_k 生成一条轨迹 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$
- 4: $g \leftarrow 0$
- 5: **for** $t = T - 1, \dots, 0$ **do**
- 6: 计算状态动作对 (s_t, a_t) 的累计折扣回报 g
- 7: $g \leftarrow r_t + \gamma g$
- 8: **if** s_t not in $(s_0, \dots, s_{t-1})$ **then**
- 9: $G(s_t, a_t) \leftarrow G(s_t, a_t) + g$
- 10: $Num(s_t, a_t) \leftarrow Num(s_t, a_t) + 1$
- 11: $Q(s_t, a_t) \leftarrow G(s_t, a_t) / Num(s_t, a_t)$
- 12: **end if**
- 13: **end for**
- 14: 利用公式 1.39 更新策略, 得到 π_{k+1}
- 15: **end for**
- 16: **return** $\pi_*(s)$

首次访问蒙特卡洛伪码

时序差分方法

蒙特卡洛方法：从完整轨迹中估计值

$$V(s) \leftarrow \text{Mean}\{G_{t:T} | S_t = s\}$$

缺点：

- 只能在整个episode结束时更新策略；
- 策略收敛缓慢；
- 值估计方差较大。

动态规划方法：用下一步值计算当前值

$$v(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v(s')$$

缺点：

- 要求下一步状态准确值已知；
- 要求转移函数 $\mathcal{P}_{ss'}^a$ 已知。

时序差分方法：用下一步估计值估计当前值

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

综合以上两者思想，实现优势互补：

- 可以在episode中更新策略；
- 无需已知转移函数 $\mathcal{P}_{ss'}^a$ 或准确值函数；
- 值估计方差较小。

时序差分方法

蒙特卡洛方法：用整个episode的累积回报 G_t 更新 $V(S_t)$

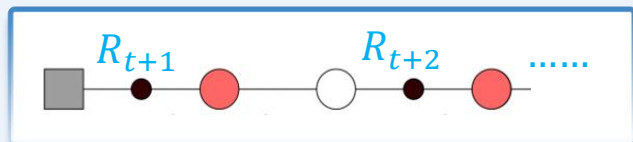
$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

更新目标： $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^T R_{t+T}$

回顾贝尔曼等式的推导

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^T R_{t+T} = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \approx R_{t+1} + \gamma V(S_{t+1})$$

从而： $G_t \approx R_{t+1} + \gamma V(S_{t+1})$



时序差分方法：将 $R_{t+2} + \gamma R_{t+3} + \dots$ 替换为 $V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

- $R_{t+1} + \gamma V(S_{t+1})$ 被称为 **TD 目标**
- $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ 被称为 **TD 误差**

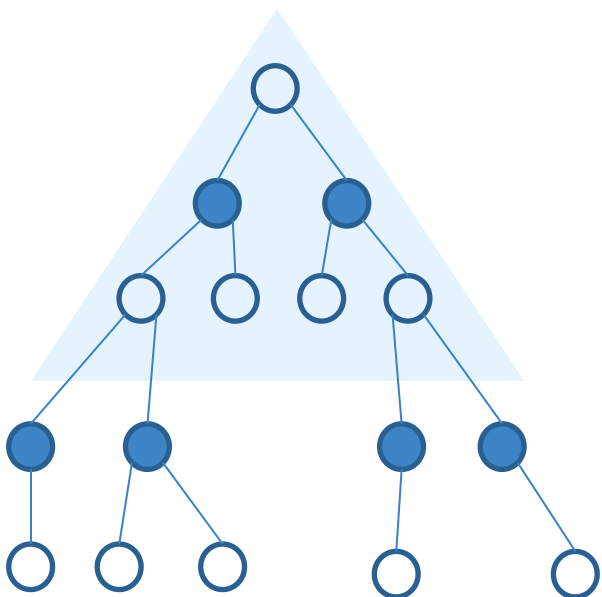


自举：用估计值 $V(S_{t+1})$ 来作为自身 $V(S_t)$ 的学习目标

时序差分方法

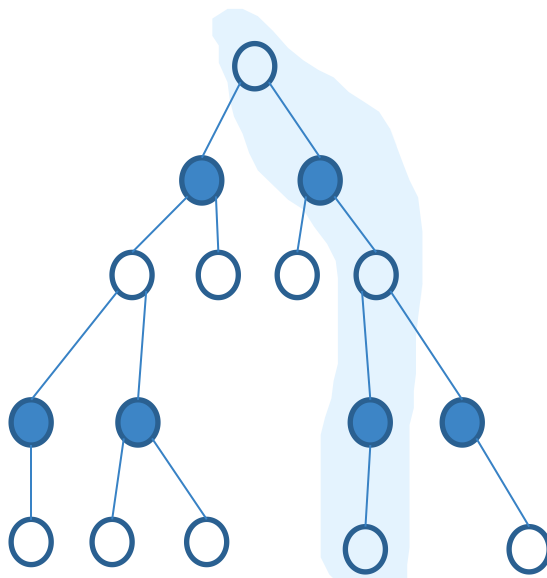
动态规划

$$v(S_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma v(S_{t+1})]$$



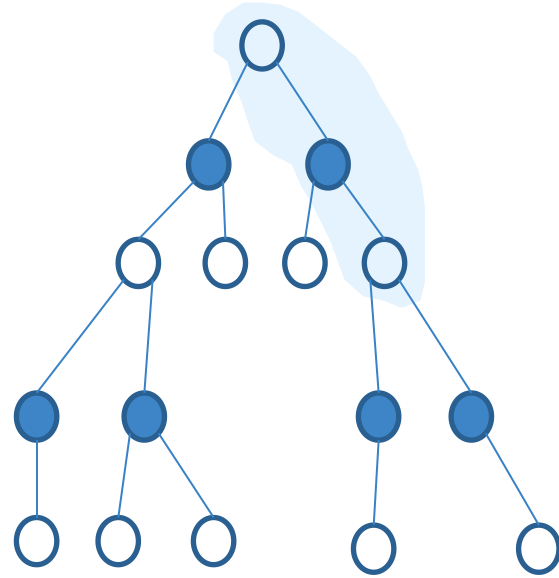
蒙特卡洛

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$



时序差分

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



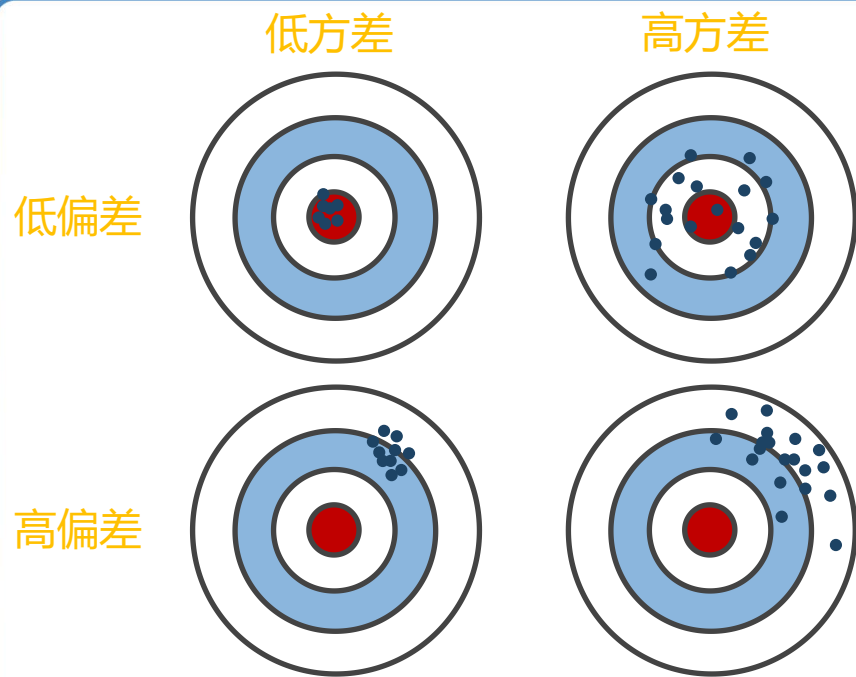
时序差分方法

偏差

- MC: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ 是价值函数 $v_\pi(S_t)$ 的**无偏**估计
- TD: 由于 $G_t \approx R_{t+1} + \gamma V(S_{t+1})$, 因此 TD 目标 $R_{t+1} + \gamma V(S_{t+1})$ 是价值函数 $v_\pi(S_t)$ 的**有偏**估计

方差

- MC: 目标依赖于**多个**随机变量 $S_t, A_t, R_t, \dots, S_T$, 方差**更大**
- TD: $R_{t+1} + \gamma V(S_{t+1})$ 只依赖于**两个**随机变量, 方差**更小**



总的来说, 偏差方面MC优于TD,
方差方面TD优于MC

时序差分方法

理论

- 蒙特卡洛方法收敛于以下最小均方误差的解

$$\sum_{k=1}^K \sum_{t=1}^{T_k} (G_t^k - V(s_t^k))^2$$

- 时序差分方法收敛于马尔可夫模型的极大似然解

$$\hat{P}_{s,s'}^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{T_k} \mathbf{1}(s_t^k, a_t^k, s_{t+1}^k = s, a, s')$$
$$\hat{R}_{s,s'}^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{T_k} \mathbf{1}(s_t^k, a_t^k = s, a) r_t^k$$

时序差分方法

例子

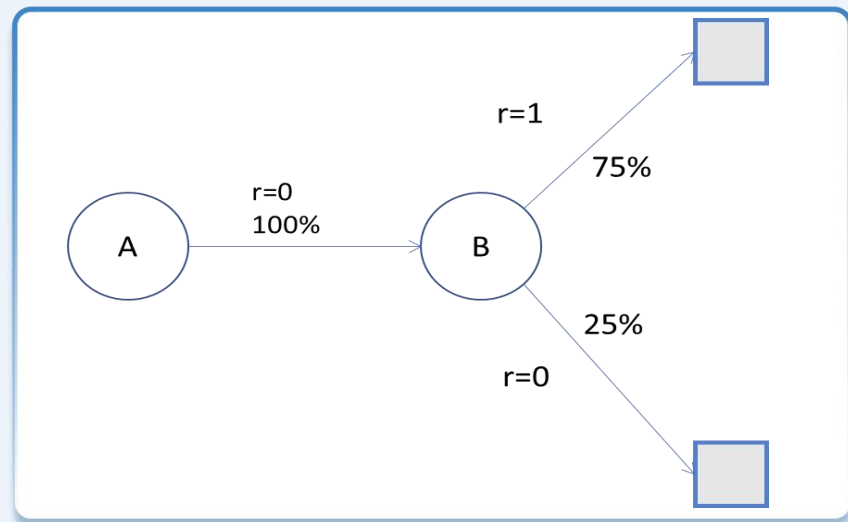
两个状态 A、B；不考虑折扣

已知：现有 8 条轨迹 $(S_1, R_1, S_2, R_2, \dots, S_T, R_T)$ ：

$(A, 0, B, 0), (B, 1), (B, 1), (B, 1), (B, 1), (B, 1), (B, 1), (B, 0)$

显然 $V(B) = 0.75$

问题：状态 A 的价值 $V(A)$ ？



解答：

- 蒙特卡洛：A 的价值为 $V(A) = 0$
- 时序差分：依据 B 的价值计算 $V(A)$ ，解得 $V(A) = 0.75$

时序差分方法

蒙特卡洛方法：方差大偏差小

VS

时序差分方法：方差小偏差大

可否在两者之间进行权衡？

两个极端

$$n = 1 \quad G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$$



时序差分

$$n = 2 \quad G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$

 $\vdots$

折中



$$n = \infty \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$



蒙特卡洛

定义 n 步回报如下：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

 n 步时序差分

定义

利用 n 步回报作为目标进行更新的方法称为 n 步时序差分学习：

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$$

时序差分方法

n步回报中, n取多少合适?

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

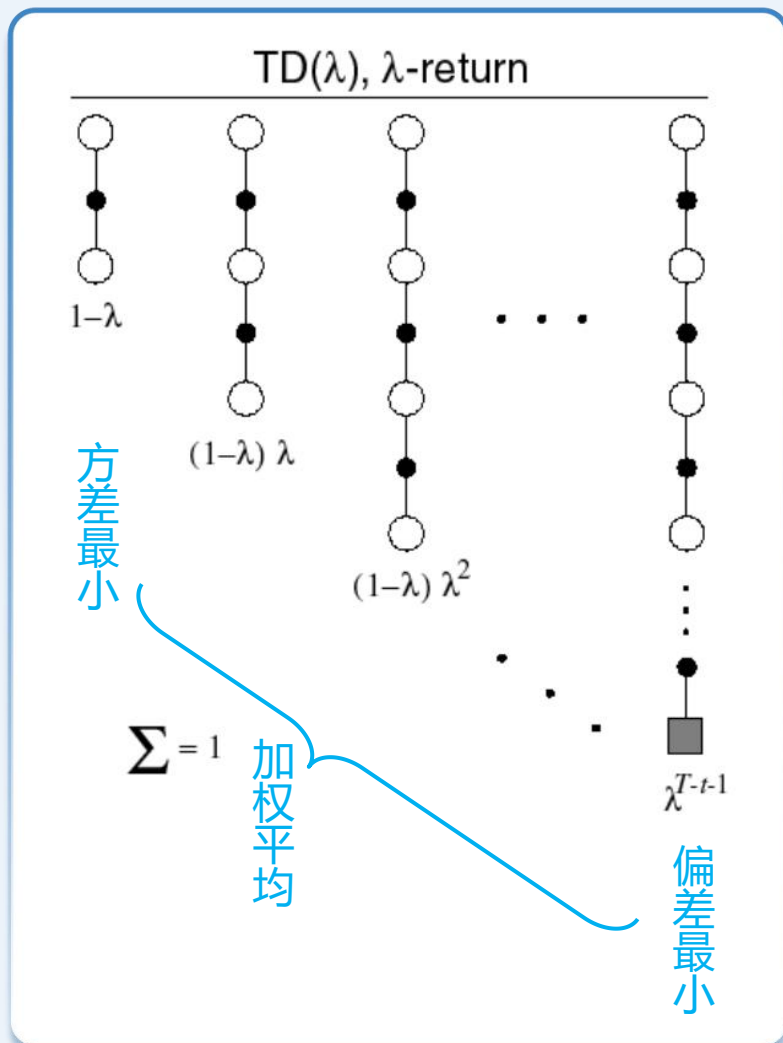
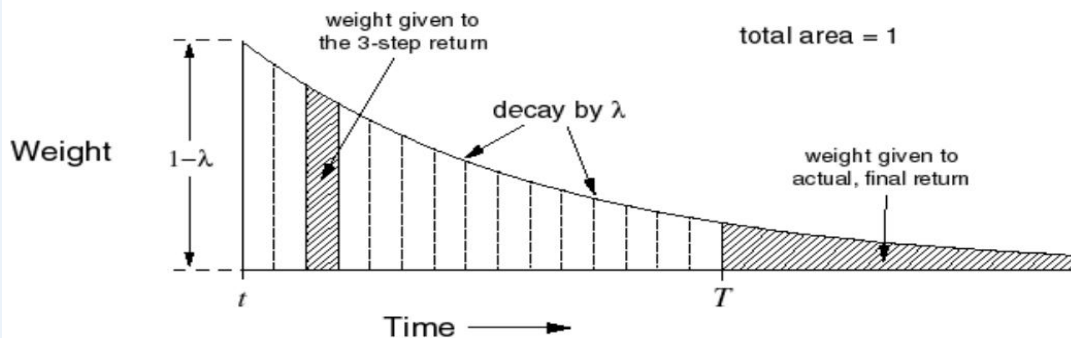
n越大 → 随机变量越多 → 方差越大, 偏差越小

- λ-回报: 使用权重 $(1-\lambda)\lambda^{n-1}$ 对多个n进行加权平均

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

定义

前向视角TD(λ)学习: $V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t))$



时序差分方法-SARSA

使用 TD 方法作为策略迭代中的策略评估方法

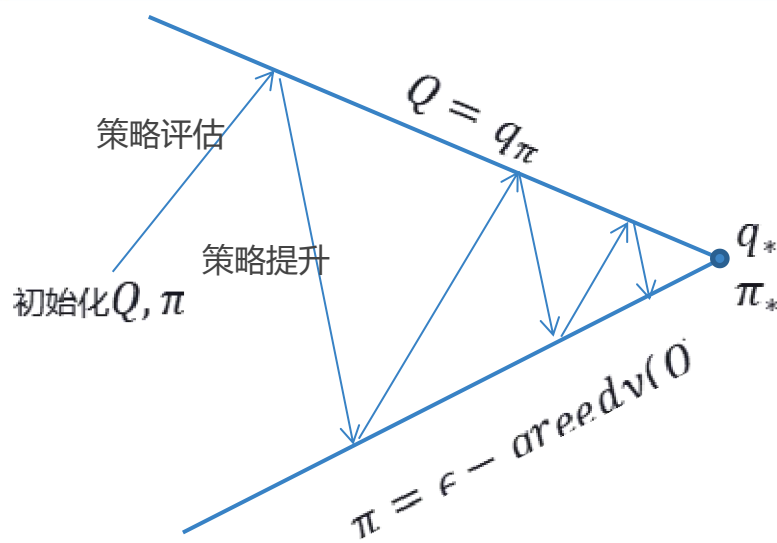
- 模型未知，因此直接估计动作价值函数

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)) \quad (*)$$

策略迭代两步走：

- 策略评估：**使用 (*) 式估计动作价值函数 $Q = q_\pi$
- 策略提升：**使用 $\epsilon - greedy$ 策略提升方法

由于(*)式使用 $\langle S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1} \rangle$ 进行更新，因此称为 **SARSA** 算法。



时序差分方法-SARSA

Algorithm 1.6 SARSA

Input: 迭代轮数 T , 更新步长 α , 衰减因子 γ , 最大探索率 ϵ_{start} , 最小探索率 ϵ_{min} , 最大交互回合数 N , 最大时间步长 T

Output: 最优值函数 Q_*

- 1: 随机初始化所有的状态动作对应的值函数 Q 。
- 2: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 3: 初始化初始状态 s_0
- 4: 在状态 s_0 利用 $\epsilon - greedy$ 策略选择动作 a_0
- 5: **for** $t = 0, 1, \dots, T$ **do**
- 6: 在状态 s_t 执行当前动作 a_t , 得到新状态 s_{t+1} 和奖励 r_t
- 7: 在新状态 s_{t+1} 利用 $\epsilon - greedy$ 策略选择动作 a_{t+1}
- 8: 利用公式 1.44 更新值函数 $Q(s_t, a_t)$
- 9: 更新探索率 ϵ
- 10: **end for**
- 11: **end for**

收敛性: 由于 SARSA 是 TD 方法的一种形式, 因此 SARSA 具有和 TD 方法相同的收敛性:

$$\begin{aligned}\sum_{t=1}^{T-1} \alpha \delta_t E_t(s) &= \alpha \sum_{t=k}^{T-1} \gamma^{t-k} \delta_t \\ &= \alpha (G_k - (S_k))\end{aligned}$$

时序差分方法-SARSA

考虑以下当 $n = 1, 2, \dots, \infty$ 时的 n 步 Q 回报：

$$\begin{aligned} n = 1 \quad q_t^{(1)} &= R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \\ n = 2 \quad q_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(S_{t+2}, A_{t+2}) \\ &\vdots \\ n = \infty \quad q_t^{(\infty)} &= R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T \end{aligned}$$

定义 n 步 Q 回报

$$q_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

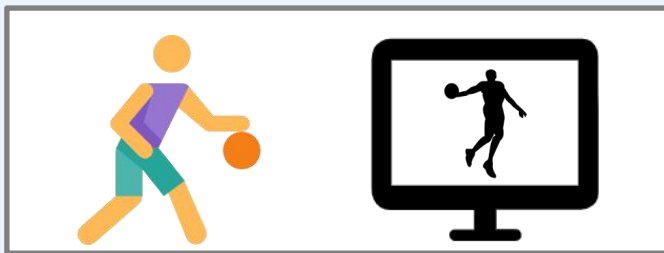
n 步 SARSA 以 n 步 Q 回报为目标更新动作价值函数 $Q(s, a)$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(q_t^{(n)} - Q(S_t, A_t))$$

时序差分方法

同策略 (on-policy)

- 边采样边学习
- 行为策略与目标策略**一致**



优势:

- 保持充分探索环境，并同步学习最优策略
- 可以通过观测其他智能体的行为进行学习
- 重用旧策略 $\pi_1, \pi_2, \dots, \pi_{t-1}$ 的经验

异策略 (off-policy)

- 行为策略与目标策略**不一致**
- 利用行为策略 $\mu(a|s)$ 采集经验 $S_1, A_1, R_2, \dots, S_T \sim \mu$
- 随后利用经验 $S_1, A_1, R_2, \dots, S_T$ 更新目标策略 π



时序差分方法-Q Learning

考虑动作价值函数 $Q(s, a)$ 的异策略学习

- 根据贝尔曼方程，我们可以得到如下公式：
$$Q_{\pi}(S_t, A_t) = \mathbb{E}_{\pi}[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})]$$

其中 R_{t+1} 为即时奖励， γ 为折扣因子， S_{t+1} 为下一状态， A_{t+1} 为策略 π 在状态 S_{t+1} 下所要采取的动作。

- 如果我们令策略 π 为依据动作价值函数 $Q(s, a)$ 的贪婪策略，那么有

$$\begin{aligned}\pi(s) &= \arg \max_a Q(s, a) \\ Q_{\pi}(S_t, A_t) &= \mathbb{E}_{\pi} \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \right]\end{aligned}$$

当给定 S_t 以及 A_t 后， R_{t+1} 与 S_{t+1} 仅由环境决定，而与所采取的策略无关。因此，当我们通过某个策略采集到样本 $\langle S_t, A_t, R_{t+1}, S_{t+1} \rangle$ 后，可直接以 $R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$ 作为目标更新贪婪策略 π 的动作价值函数 $Q_{\pi}(S_t, A_t)$ ，而无需进行重要性采样。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$$

时序差分方法-Q Learning

在实际应用时，通常使用 ϵ -贪婪策略与环境交互，以充分探索环境，下面给出 Q-学习算法的伪代码。

Algorithm 1.7 Q-Learning

Input: 迭代轮数 T , 更新步长 α , 衰减因子 γ , 最大探索率 ϵ_{start} , 最小探索率 ϵ_{min} , 最大交互回合数 N , 最大时间步长 T

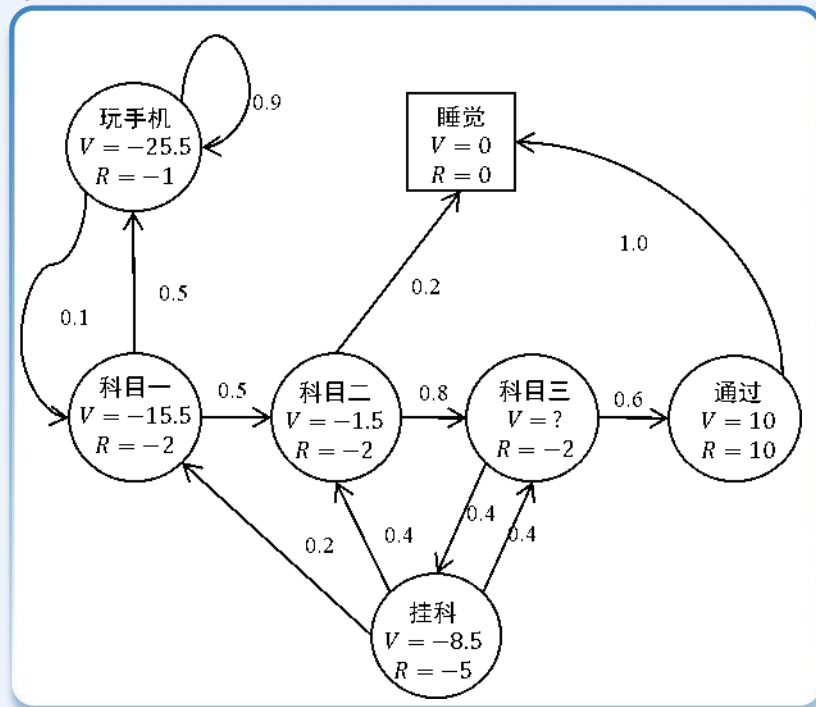
Output: 最优值函数 Q_*

- 1: 随机初始化所有的状态动作对应的值函数 Q 。
- 2: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 3: 初始化初始状态 s_0
- 4: **for** $t = 0, 1, \dots, T$ **do**
- 5: 在状态 s_t 利用 $\epsilon - greedy$ 策略选择动作 a_t
- 6: 在状态 s_t 执行当前动作 a_t , 得到新状态 s_{t+1} 和奖励 r_t
- 7: 利用公式 1.47 更新值函数 $Q(s_t, a_t)$
- 8: 更新探索率 ϵ
- 9: **end for**
- 10: **end for**

MDP实例

考虑如图所示的MDP：一个学生需要学习三个科目，然后通过测验；不过也有可能只学完两个科目之后直接睡觉，或者在学习时玩手机；一旦挂科，有可能需要重新学习某些科目。其中，椭圆表示普通状态，每一条线上的数字表示从一个状态跳转到另一个状态的概率，R代表奖励，方块表示终止（terminal）状态：

- 给定折扣因子 $\gamma=0.5$ ，计算轨迹“科目一，科目二，科目三，通过，睡觉”以及轨迹“科目一，玩手机，玩手机，科目一，科目二，睡觉”的回报值。
- 给定折扣因子 $\gamma=1$ ，求所处状态“科目三”的V值。



MDP实例

(1) 给定折扣因子 $\gamma=0.5$ ，计算轨迹“科目一，科目二，科目三，通过，睡觉”以及轨迹“科目一，玩手机，玩手机，科目一，科目二，睡觉”的回报值。

轨迹“科目一，科目二，科目三，通过，睡觉”：

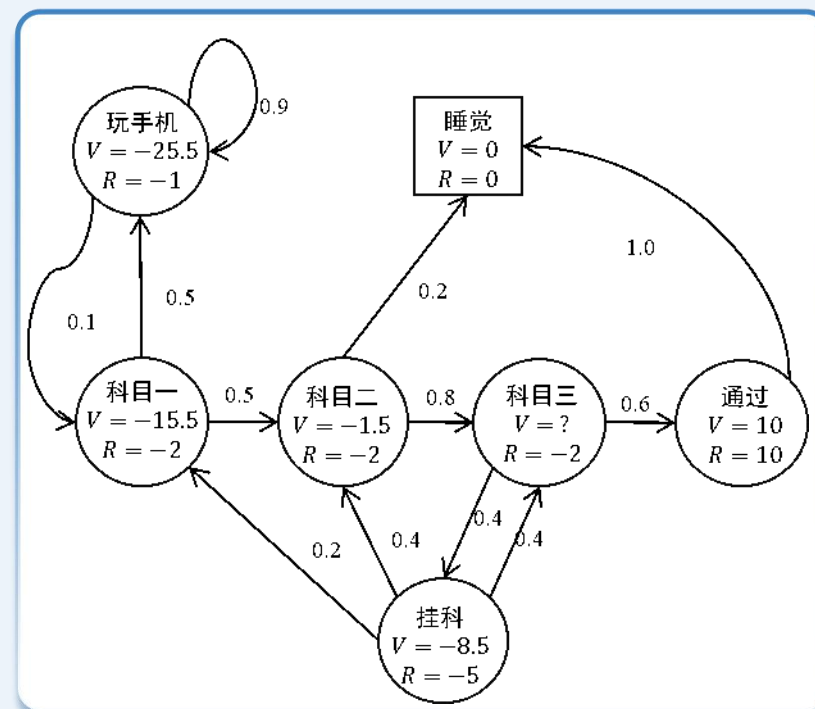
$$G_1 = -2 + 0.5 \times -2 + 0.5^2 \times -2 + 0.5^3 \times 10 + 0.5^4 \times 0 = -2.25$$

轨迹“科目一，玩手机，玩手机，科目一，科目二，睡觉”

$$G_2 = -2 + 0.5 \times -1 + 0.5^2 \times -1 + 0.5^3 \times -2 + 0.5^4 \times -2 + 0.5^5 \times 0 = -3.125$$

(2) 给定折扣因子 $\gamma=1$ ，求所处状态“科目三”的V值。

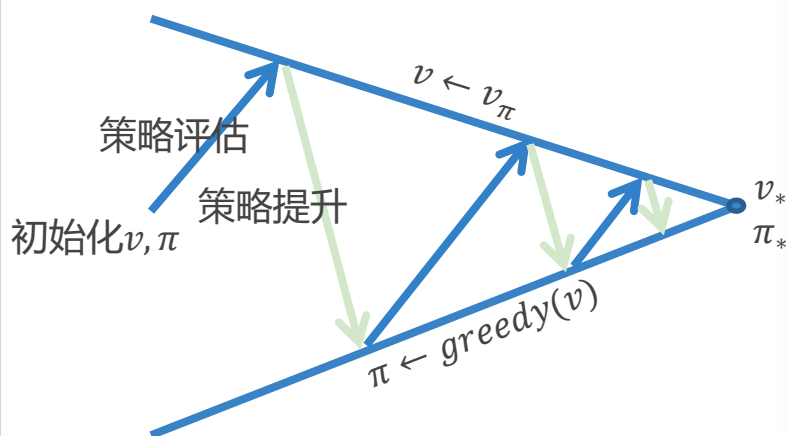
$$V = -2 + 0.6 \times 10 + 0.4 \times -8.5 = 0.6$$



本节小结

期望更新（动态规划）

- 策略评估
- 策略改进
- 策略迭代



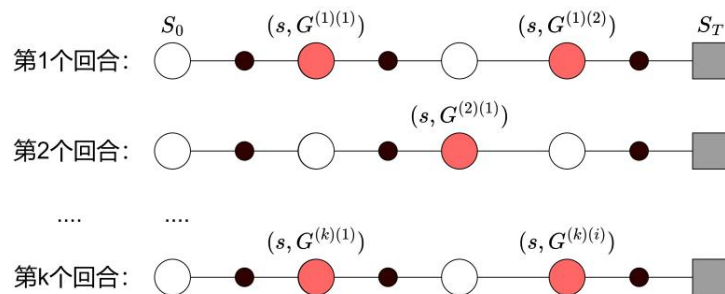
采样更新（蒙特卡洛）

- 首次访问蒙特卡洛

$$V_N(s) = \frac{G^{(1)(1)}(s) + G^{(2)(1)}(s) + \dots + G^{(N)(1)}(s)}{N}$$

- 每次访问蒙特卡洛

$$V_{N \times M}(s) = \frac{G^{(1)(1)}(s, a) + \dots + G^{(N)(1)}(s, a) + \dots + G^{(N)(M)}(s, a)}{N \times M}$$



采样更新（时序差分）

- 时序差分方法

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

- SARSA

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

- Q-Learning

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

- 同策略 (on-policy)
- 异策略 (off-policy)

深度强化学习

表格型强化学习方法：需要将环境的状态和动作表示为一个有限的表格（如 Q 表）来存储每个状态-动作对的价值或策略，具有

- **理论简单、易于实现：**表格型方法如 Q-Learning 和 SARSA 的算法简单，易于入门，适合用于强化学习基础理论的教学和研究；
- **敛性强：**对于有限状态和动作空间的问题，只要满足一定条件（如探索足够多次），算法可以收敛到最优策略；
- **对小规模问题效果优异：**在状态和动作空间有限的情况下，表格型方法往往能高效地找到最优解；
- **调试方便：**表格的形式直观，可直接观察状态价值或策略的变化过程，便于调试和分析问题。

深度强化学习

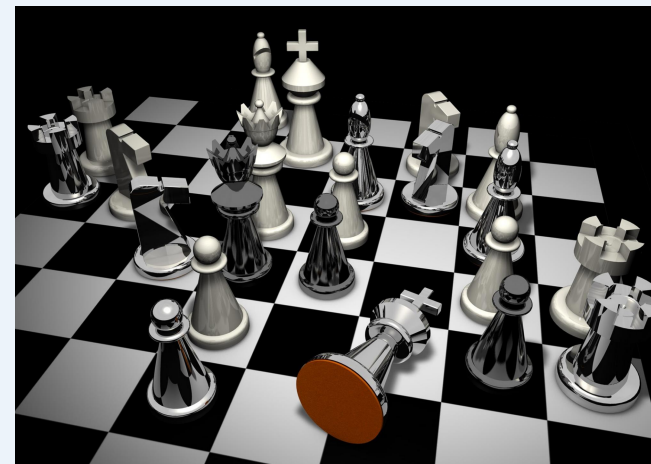
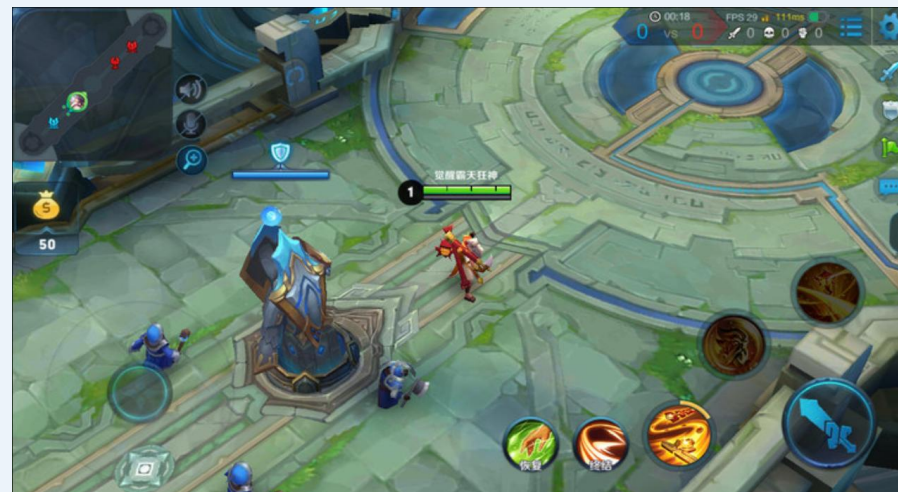
然而，表格型强化学习方法面临着大量的问题：

- **扩展性差**：状态和动作空间较大时，表格的大小会急剧增长（“维度灾难”），导致存储和计算成本过高，难以应用于高维或连续空间。
- **对复杂环境不适用**：表格型方法无法直接处理复杂的、具有连续状态或动作空间的问题，例如机器人控制和自动驾驶中的应用场景。
- **缺乏泛化能力**：表格型方法对未探索到的状态无法进行泛化处理，表现为对未见状态完全缺乏经验，导致策略表现差。
- **存储开销大**：随着状态-动作空间增大，存储需求成指数增长，资源消耗显著增加。
- **采样效率低**：在高维空间中，穷举式的探索会导致算法采样效率极低，难以收敛。

深度强化学习

如何将强化学习算法扩展到大规模状态动作空间问题上？

- 象棋：约 10^{120} 个状态
- 围棋：约 10^{170} 个状态
- 王者荣耀、自动驾驶：连续状态空间



深度强化学习

深度学习：是机器学习的一个子领域，通过利用神经网络（特别是深层神经网络）来模拟人类大脑的思维和学习过程，从海量数据中自动提取特征并完成任务。

深度强化学习（Deep Reinforcement Learning, DRL）：是强化学习和深度学习的结合，通过深度神经网络（DNN）处理复杂环境中的强化学习问题：

- **深度神经网络**：能处理图像、文本等复杂的感知数据或者高维连续的向量数据
- **强化学习**：处理序列决策问题

深度强化学习通常使用深度神经网络表征状态值函数、状态动作值函数、策略函数、环境模型等。

基于值的RL

- 基于值函数的深度强化学习算法是由 Q-Learning 算法结合深度神经网络衍生的一类算法，主要包括 DQN 算法及其改进算法 Double DQN、Dueling DQN、Rainbow DQN。
- Q-Learning 算法直接结合神经网络的算法称为 **Naïve DQN**。

通过拟合数据集得到最优动作价值函数

Naïve DQN

使用某个策略采样数据集

$$\{\langle s_1, a_1, s'_1, r_1 \rangle, \dots, \langle s_n, a_n, s'_n, r_n \rangle\}$$

$$y_i = r_i + \gamma \max_{a'} Q_w(s'_i, a')$$

$$\mathbf{w} \leftarrow \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^n \| Q_w(s_i, a_i) - y_i \|^2$$

误差 ε

$$\varepsilon = \frac{1}{2} \mathbb{E}_{(s,a,s',r) \sim \mathcal{B}} \left[\left(Q_w(s, a) - \left[r + \gamma \max_{a'} Q_w(s', a') \right] \right)^2 \right]$$

当 $\varepsilon = 0$ 时，有

$$Q_w(s, a) = r + \gamma \max_{a'} Q_w(s', a')$$

此时为最优动作价值函数 $Q_w(s, a) = q_*(s, a)$ ，可由此求得最优策略 π_* ：

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

基于值的RL

- Q -Learning算法直接结合神经网络的算法称为 **Naïve DQN**。

在线版本的Naïve DQN算法

Naïve DQN

使用某个策略采样数据集

$$\{\langle s_1, a_1, s'_1, r_1 \rangle, \dots, \langle s_n, a_n, s'_n, r_n \rangle\}$$

$$y_i = r_i + \gamma \max_{a'} Q_w(s'_i, a')$$

$$\mathbf{w} \leftarrow \operatorname{argmin} \frac{1}{n} \sum_{i=1}^n \|Q_w(s, a_i) - y_i\|^2$$

在线 Naïve DQN算法

采取某个动作 a_i ，观察得到 $\langle s_i, a_i, s'_i, r_i \rangle$

$$y_i = r_i + \gamma \max_{a'} Q_w(s'_i, a')$$

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha (Q_w(s_i, a_i) - y_i) \nabla_{\mathbf{w}} Q(s_i, a_i)$$

生成样本

更新价值函数

提升策略

$$\pi(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in \mathcal{A}} Q_w(s, a) \\ 0 & \text{otherwise} \end{cases}$$

基于值的RL

- Q -Learning算法直接结合神经网络的算法称为 **Naïve DQN**。

在线版本的Naïve DQN算法

Algorithm 1.8 Naive DQN 算法

Input: 迭代轮数 T , 学习率 α , 衰减因子 γ , 最大探索率 ϵ_{start} , 最小探索率 ϵ_{min} , 最大交互回合数 N , 最大时间步长 T

Output: 最优值函数 Q_*

- 1: 随机初始化 Q 值网络参数 ϕ 。
- 2: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 3: 重置环境并获取环境的初始状态 s_0
- 4: **for** $t = 0, 1, \dots, T$ **do**
- 5: 在状态 s_t 利用 $\epsilon - greedy$ 策略选择动作 a_t
- 6: 在状态 s_t 执行当前动作 a_t , 得到新状态 s_{t+1} 和奖励 r_t
- 7: 使用公式 1.50 更新神经网络参数 ϕ
- 8: 更新探索率 ϵ
- 9: **end for**
- 10: **end for**

生成样本

更新价值函数

提升策略

$$\pi(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q_w(s, a) \\ 0 & \text{otherwise} \end{cases}$$

基于值的RL

Naïve DQN: 样本之间存在较高关联性

在线 Naïve DQN算法

采取某个动作 a_i , 观察得到 $\langle s_i, a_i, s'_i, r_i \rangle$

$$y_i = r_i + \gamma \max_{a'} Q_w(s'_i, a')$$

$$w \leftarrow w - \alpha (Q_w(s_i, a_i) - y_i) \nabla_w Q(s_i, a_i)$$

时间上相近的状态之间具有极高的关联性,
不利于训练的稳定性

相邻的几个帧的画面非常相似



基于值的RL

Naïve DQN: 样本之间存在较高关联性

使用经验回放池打破样本相关性

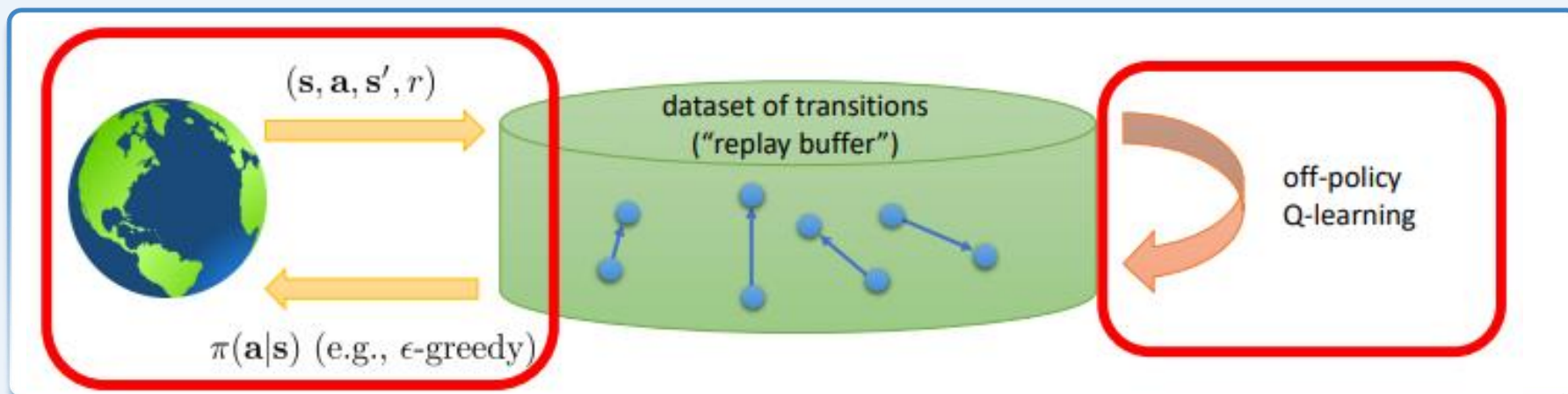
基于经验回放池的 Q-learning

与环境交互获取数据 $\{(s, a, s', r)\}$, 并存入缓存 B

从缓存 B 中采样一批数据 $\{(s_1, a_1, s'_1, r_1), \dots, (s_n, a_n, s'_n, r_n)\}$

$$w \leftarrow w - \sum_{i=1}^n \alpha (Q_w(s', a') - y) \nabla_w Q(s, a)$$

样本之间不再具有较高的关联性
批量中样本能够降低梯度的方差



基于值的RL

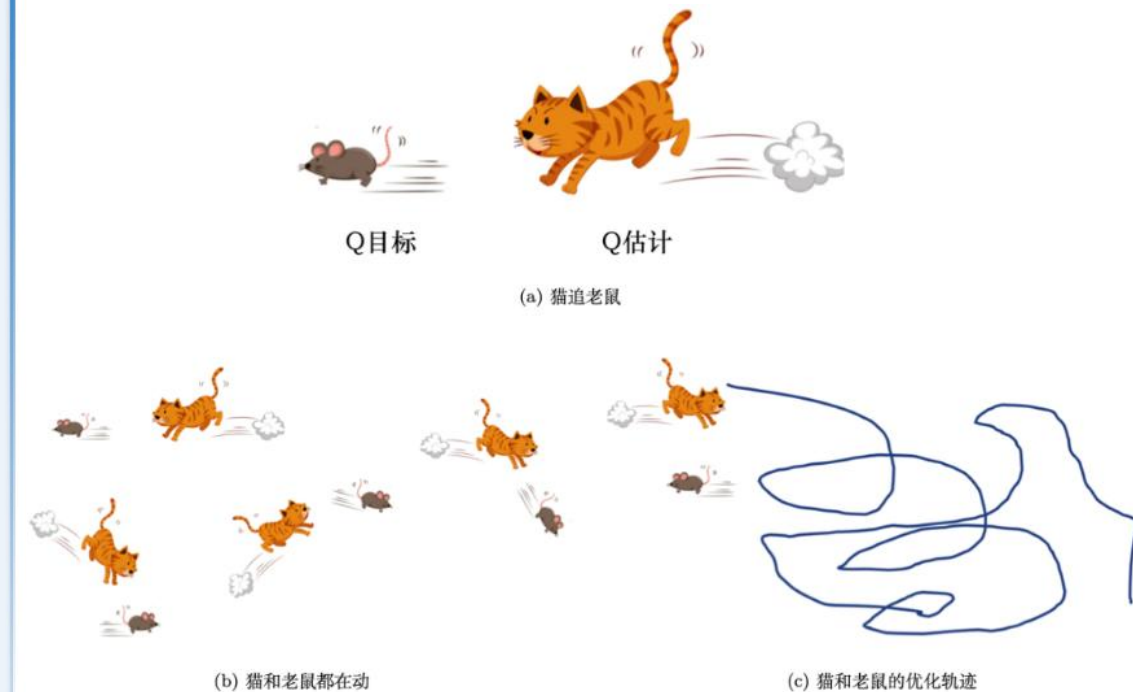
Naïve DQN: 不是梯度下降方法

采取某个动作 a_i , 观察得到 $\langle s_i, a_i, s'_i, r_i \rangle$

$$w \leftarrow w - \alpha \left(Q_w(s_i, a_i) - \left[r_i + \gamma \max_{a'} Q_w(s'_i, a') \right] \right) \nabla_w Q(s_i, a_i)$$

semi-gradient

Q价值网络更新的同时, 拟合的目标也在变动, 因此不能保证该过程的收敛性。训练过程不稳定, 甚至发生震荡。



基于值的RL

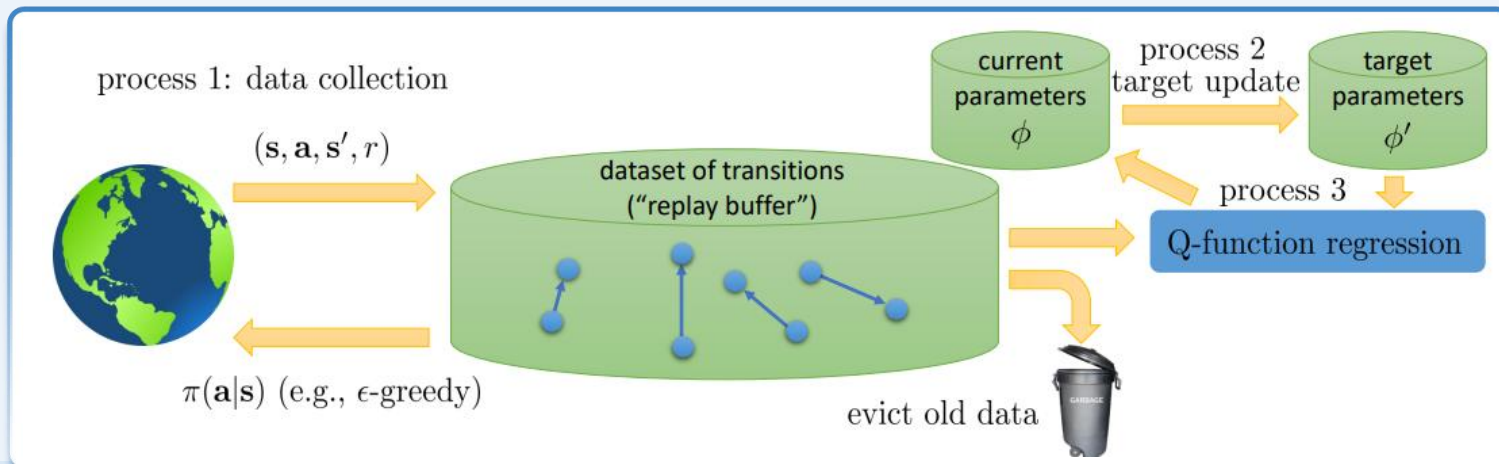
Naïve DQN: 不是梯度下降方法

使用目标网络提高值网络训练的稳定性

使用经验回放池和目标网络的 Q-learning

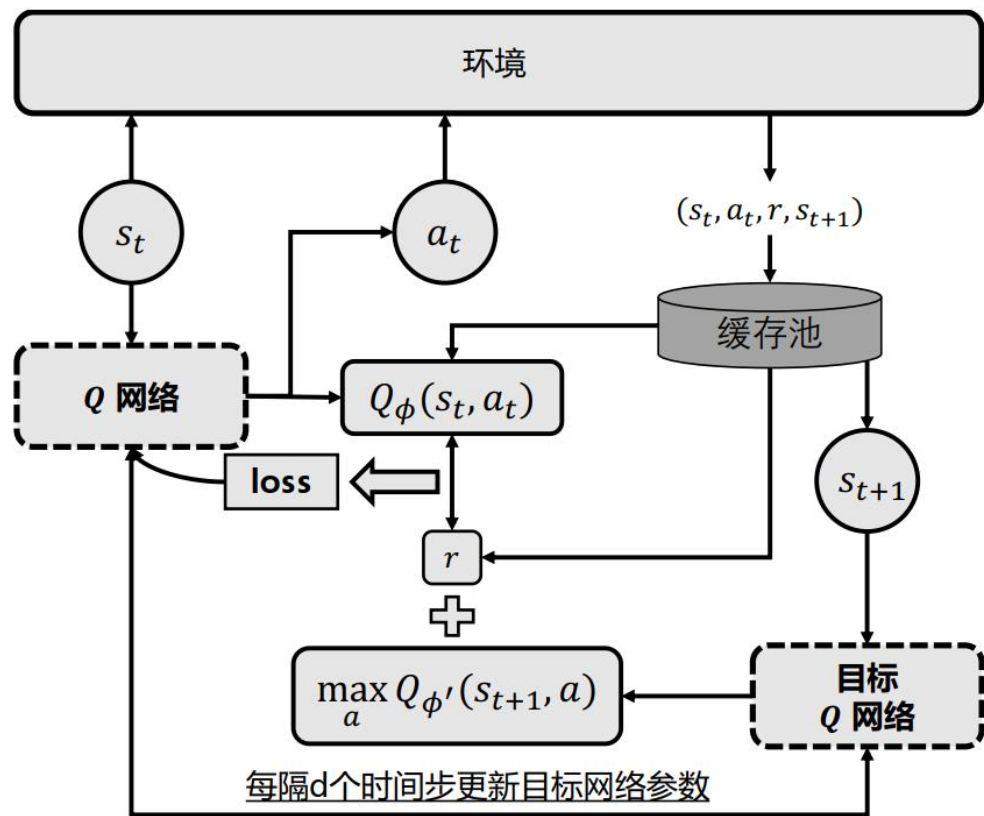
1. 设置目标网络的参数 $w' \leftarrow w$
2. 与环境交互获取数据 $\{(s, a, s', r)\}$, 并存入缓存 B
3. 从缓存 B 中采样一批数据 $\{(s_1, a_1, s'_1, r_1), \dots, (s_n, a_n, s'_n, r_n)\}$
4. $w \leftarrow w - \alpha \sum_{i=1}^n \left(Q_w(s_i, a_i) - \left[r_i + \gamma \max_{a'} Q_{w'}(s'_i, a') \right] \right) \nabla_w Q(s_i, a_i)$

使用目标网络计算
目标值
定期更新目标网络
的参数



基于值的RL

Classic DQN: 使用经验回放池以及目标网络的DQN



Algorithm 1.9 DQN 算法

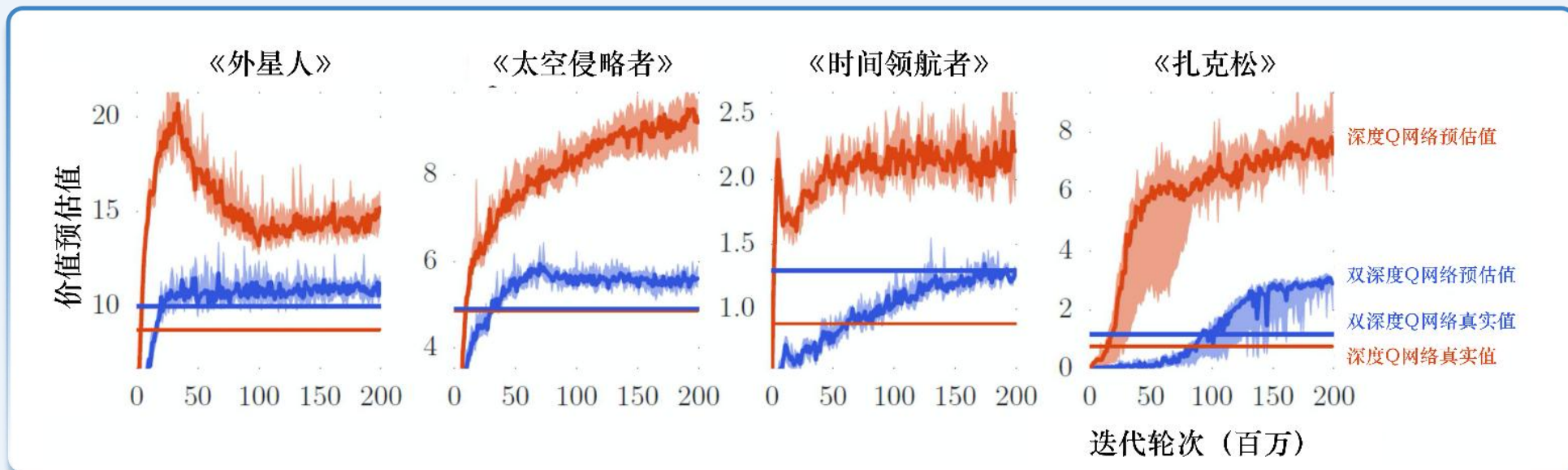
Input: 迭代轮数 T , 学习率 α , 衰减因子 γ , 最大探索率 ϵ_{start} , 最小探索率 ϵ_{min} , 最大交互回合数 N , 最大时间步长 T , 梯度更新次数 G , 经验回放池 B , 批样本大小 B , 目标网络更新间隔 d

Output: 最优值函数 Q_*

- 1: 随机初始化 Q 值网络参数 ϕ
- 2: 初始化目标值网络参数 $\phi' \leftarrow \phi$
- 3: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 4: 重置环境并获取环境的初始状态 s_0
- 5: **for** $t = 0, 1, \dots, T$ **do**
- 6: 在状态 s_t 利用 $\epsilon - greedy$ 策略选择动作 a_t
- 7: 在状态 s_t 执行当前动作 a_t , 得到新状态 s_{t+1} 和奖励 r_t
- 8: 将四元组 (s_t, a_t, r_t, s_{t+1}) 存入经验回放池 B 中
- 9: **for** 每个梯度更新步 $g = 1, \dots, G$ **do**
- 10: 从经验回放池 B 中采样一批大小为 B 数据样本
- 11: 使用公式 1.52 更新神经网络参数 ϕ
- 12: **end for**
- 13: **if** $(k \times T) + t \bmod d == 0$ **then**
- 14: 使用公式 1.53 更新目标网络参数 ϕ'
- 15: **end if**
- 16: 更新探索率 ϵ
- 17: **end for**
- 18: **end for**

基于值的RL

Classic DQN: Q 值易被高估



- 上图为 DQN 和 DDQN 应用在不同小游戏上，得到的价值的预估值及真实值对比。
- 可以看出 DQN 容易过高的估计所能得到的回报，从而收敛到一个次优解。

为什么 DQN 会高估所能得到的回报呢？

基于值的RL

Classic DQN: Q 值易被高估

- 一个数学直觉，考虑两个随机变量 X_1, X_2

$$\mathbb{E}[\max(X_1, X_2)] \geq \max(\mathbb{E}[X_1], \mathbb{E}[X_2])$$

- 不妨假设动作价值网络的估计误差为随机噪音 $X_1, X_2 \sim N(0, \varepsilon)$

$$\begin{aligned} & \mathbb{E}_X[\max(Q(s, a_1) + X_1, Q(s, a_2) + X_2)] \\ & \geq \max(\mathbb{E}_X[Q(s, a_1) + X_1], \mathbb{E}_X[Q(s, a_1) + X_2]) = \max(Q(s, a_1), Q(s, a_2)) \end{aligned}$$

- 右图中蓝色为真实价值，绿色为网络对价值的估计误差。
- 算法总是选择价值最高的动作作为目标进行更新，因此存在被高估的问题。



基于值的RL

Double DQN: 选择动作和计算价值不使用同一个网络

- DQN 计算目标值时, 基于 Q_w 选择最优动作, 再使用 Q_w 计算目标值

$$\max_{a'} Q_w(s', a') = Q_w(s', \arg\max_{a'} Q_w(s', a'))$$

通过 Q_w 计算价值 通过 Q_w 选择动作

- Double DQN 使用两个网络分别进行动作选择与目标值计算

$$Q_{w_1} \leftarrow r + \gamma Q_{w_2}(s', \arg\max_{a'} Q_{w_1}(s, a))$$

- 如果两个网络的误差不同, 则可以在一定程度上解决问题。

基于值的RL

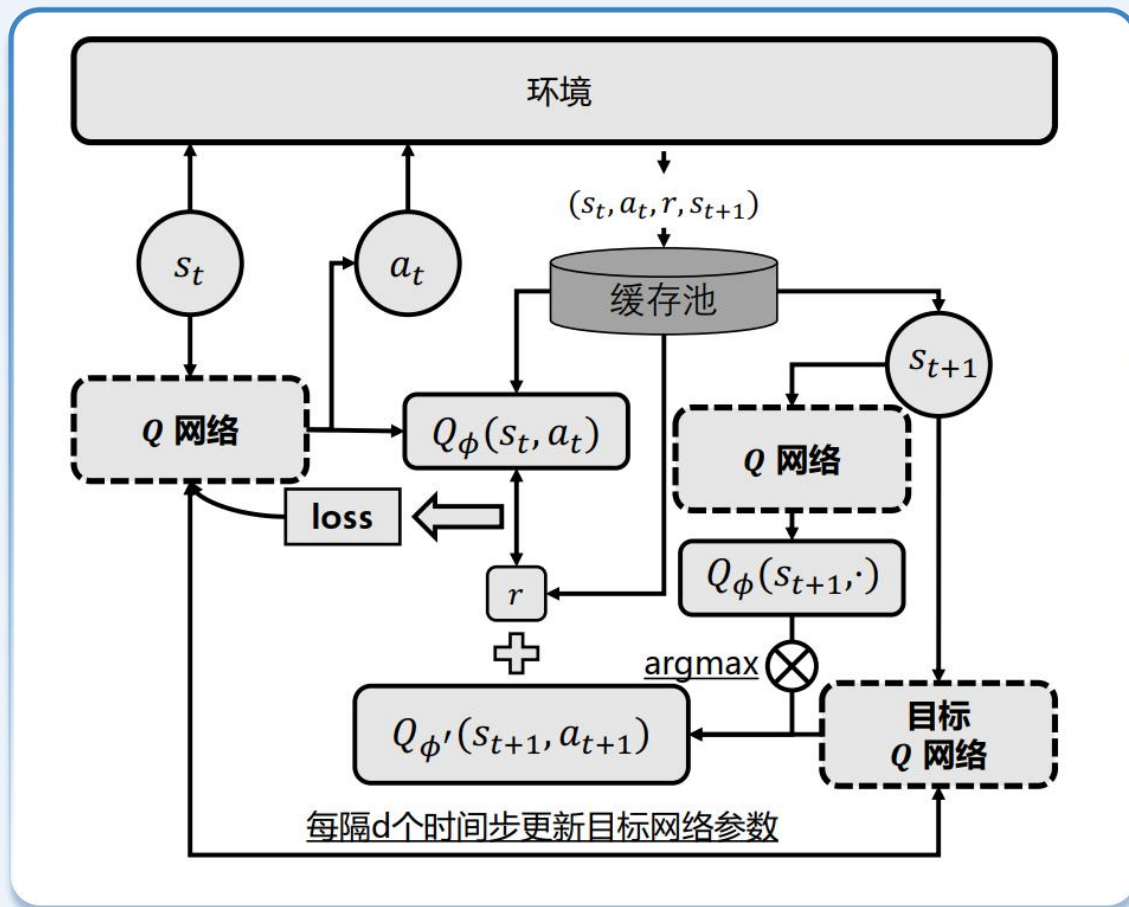
Double DQN: 选择动作和计算价值不使用同一个网络

- DQN 中的当前网络 Q_w 与目标网络 $Q_{w'}$
- DQN 中的目标值计算:

$$y = r + \gamma Q_{w'}(s', \operatorname{argmax}_{a'} Q_{w'}(s', a'))$$

- Double DQN 中的目标值计算:

$$y = r + \gamma Q_{w'}(s', \arg\max_{a'} Q_w(s', a'))$$



使用当前网络选择动作

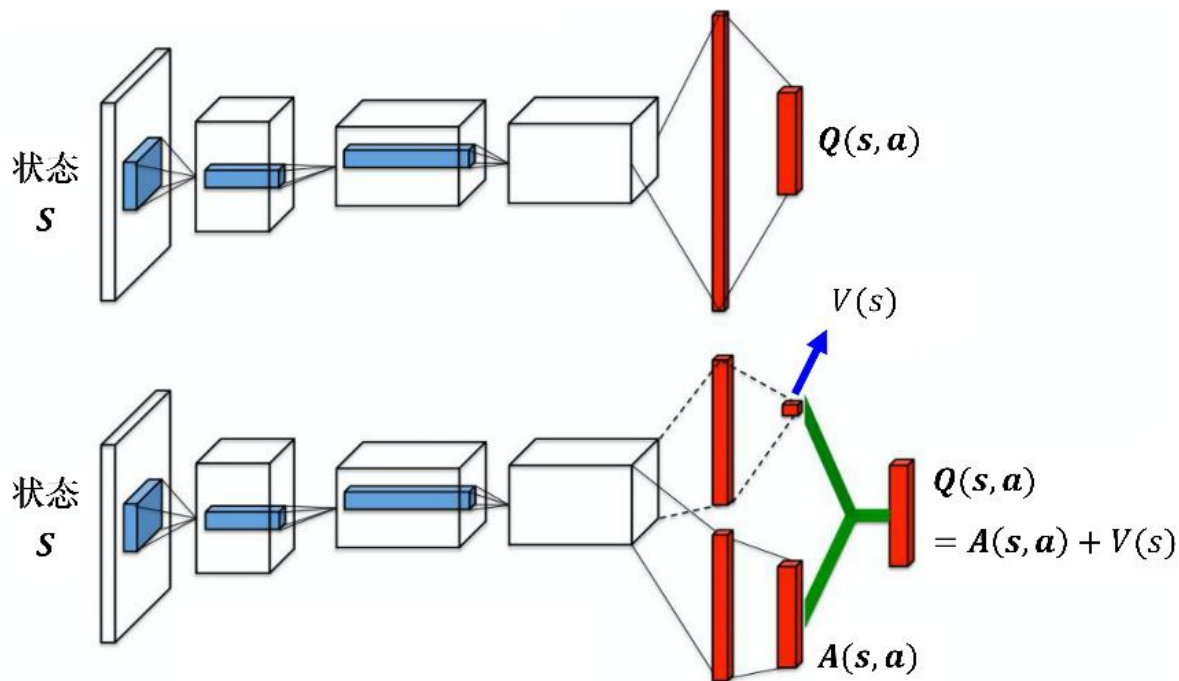
使用目标网络估计价值

基于值的RL

Dueling DQN: 尝试将原来的 Q 网络拆分为两个部分

$$Q(s, a) = V(s) + A(s, a)$$

- 该网络输出两个量:
 - ✓ $V_{\theta}(s)$: 状态 s 的平均价值
 - ✓ $A_{\phi}(s, a)$: 动作 a 优势价值

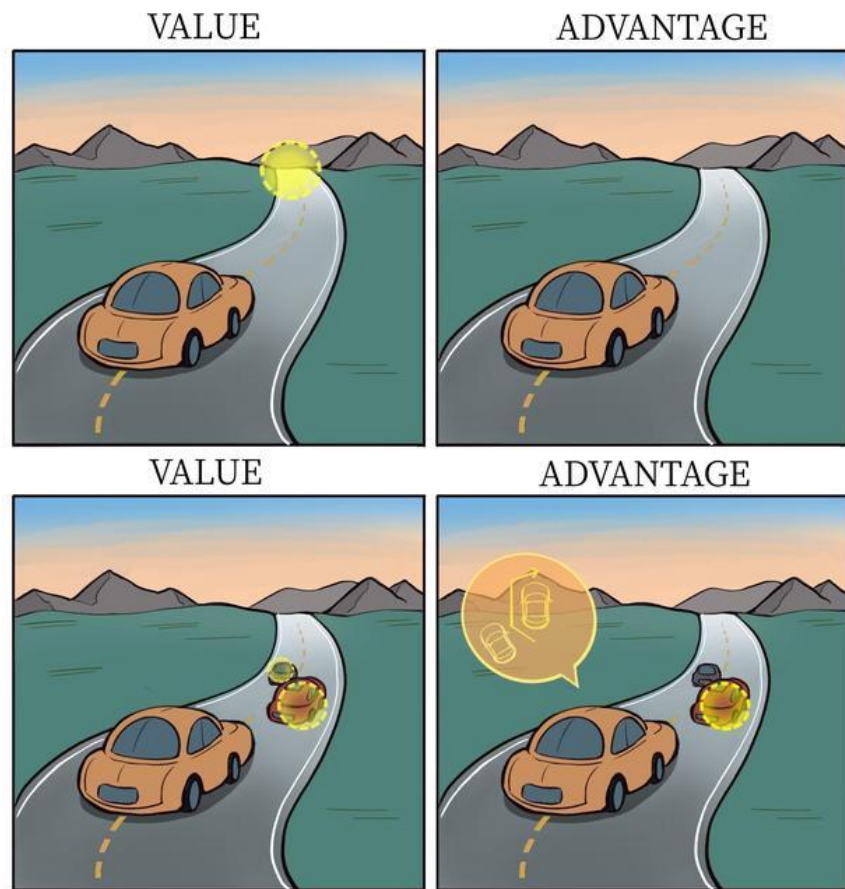


基于值的RL

Dueling DQN: 尝试将原来的 Q 网络拆分为两个部分

优势:

- 分辨当前价值是由状态还是动作带来的, 从而进行有针对性的更新, 增加样本的利用率
- 右图展示了价值函数和优势函数关注的区域
 - ✓ 当前方无车时, 不同动作的优势价值 $A_{\phi}(s, a)$ 应无明显差异。
 - ✓ 而当前方有车时, 不同动作应有不同的优势价值 $A_{\phi}(s, a)$ 。



价值函数和优势函数关注的区域

基于值的RL

Dueling DQN: 尝试将原来的 Q 网络拆分为两个部分

训练出来的 “ $V(s)$ ” 是我们想要的吗?

- 一种可能是 “ $V(s)$ ” 总是输出 0, 而 “ $A(s, a)$ ” 输出 $Q(s, a)$
- 由优势函数定义 $A_*(s, a) = Q_*(s, a) - V_*(s)$ 及 $V_*(s) = \max_{a'} Q_*(s, a')$, 易知下式成立:

$$\max_{a'} A_*(s, a') = \max_{a'} Q_*(s, a') - V_*(s) = 0$$

$$Q_*(s, a) = V_*(s) + A_*(s, a) - \max_{a'} A_*(s, a')$$

- 在计算时使用 $Q_{\theta, \phi}(s, a) = V_{\theta}(s) + A_{\phi}(s, a) - \max_{a'} A_{\phi}(s, a')$, 而非 $Q_{\theta, \phi}(s, a) = V_{\theta}(s) + A_{\phi}(s, a)$, 能够使得 $\max_{a'} A_{\phi}(s, a')$ 在收敛时趋近于零。

- 为了回传梯度, 在实际应用时一般使用平均替代最大化操作, 所以最终的Q值表示为:

$$Q_{\theta, \phi}(s, a) = V_{\theta}(s) + A_{\phi}(s, a) - \sum_a A_{\phi}(s, a') / |A|$$

基于值的RL

Rainbow DQN: 一种整合多种 DQN 改进技术的强化学习算法

- **Double DQN**: 通过分离动作选择和动作评估解决 DQN 中的过估计偏差问题;
- **Dueling DQN**: 通过引入 $V(s)$ 和 $A(s, a)$ 的分解表示改进 Q 值估计的效率;
- **优先经验回放**: 通过为经验样本分配优先级, 优先学习关键的样本, 从而提高数据利用效率;
- **多步学习**: 使用 n-步回报能够更快地传播奖励信号, 从而提升收敛速度;
- **分布式学习**: 利用分布式学习对价值分布进行建模, 从而能更精确地捕捉环境中随机性的影响;
- **噪声网络**: 在神经网络中引入参数化噪声, 使得策略具有更为灵活的探索能力;
- **目标网络的软更新**: 采用软更新方式更新目标网络参数, 避免更新过快导致的不稳定性。

基于值的RL

优先经验回放 (Prioritized Experience Replay)

- 具有较高 TD 误差的样本应该给予较高的优先级
- 方法一：采样第 t 个样本的概率 p_t 正比于 TD 误差 δ_t

$$p_t \propto |\delta_t| + \epsilon$$

ϵ 为一个很小的正数，防止采样概率为零

- 方法二：采样第 t 个样本的概率 p_t 反比于 TD 误差在全体样本中从大到小的位次 $rank(t)$

$$p_t \propto \frac{1}{rank(t)}$$

较方法一对异常点更鲁棒，异常点 TD 误差过大或过小对 $rank(t)$ 没有太大影响。

基于值的RL

优先经验回放 (Prioritized Experience Replay)

- 采用经验回放的目的之一就是打破样本之间的关联性
- 而优先级采样重新引入了样本关联性
- 引入参数 $\beta \in [0,1]$ 调整各个样本的学习率 α_t ，在二者之间做权衡

$$\alpha_t \leftarrow \alpha \cdot (np_t)^{-\beta}$$

其中 n 为样本数目

均匀采样时 $p_1 = p_2 = \dots = p_n = \frac{1}{n}$, $(np_t)^{-\beta} = 1$, 所有样本使用相同的学习率 α

基于优先级采样时, 具有较高优先级的样本使用较低的学习率 (p_t 越大, $\alpha \cdot (np_t)^{-\beta}$ 越小)

优先经验回放重新引入样本相关性

基于AC的RL

- **基于值的强化学习算法**（如 DQN 算法）难以直接应用于高维离散动作空间、连续动作空间、离散连续混合的动作空间。这些场景中，动作空间的维度可能极高甚至是无限大，因此直接枚举或计算每个动作的值函数所带来的计算成本十分高昂，同时在连续动作空间中也无法实现枚举所有可能的动作。
- **基于策略的强化学习算法（Policy-based）** 使用神经网络直接拟合策略函数。策略网络用于在给定状态 s 下生成对应的策略 $\pi(a|s)$ 。对于离散动作空间的场景，策略网络通常输出所有离散动作的概率分布，通过概率采样得到最终和环境交互的动作。对于连续动作空间的场景，策略网络通常输出一个多维高斯分布的均值和方差，通过在高斯分布中采样得到最终和环境交互的动作。
- **基于 Actor-Critic 的强化学习算法（或者称为基于演员-评论家算法）** 使用两个不同的网络，一个策略网络（Actor）和一个值网络（Critic），分别用于策略生成和策略评估。

基于AC的RL

基于值的强化学习算法

- 学习值函数
- 利用值函数导出策略
- 更高的样本训练效率
- 通常仅适用于具有离散动作的环境

基于策略的强化学习算法

- 不需要值函数
- 直接学习策略
- 在高维或连续动作空间场景中更加高效
- 适用任何动作类型的场景
- 容易收敛到次优解

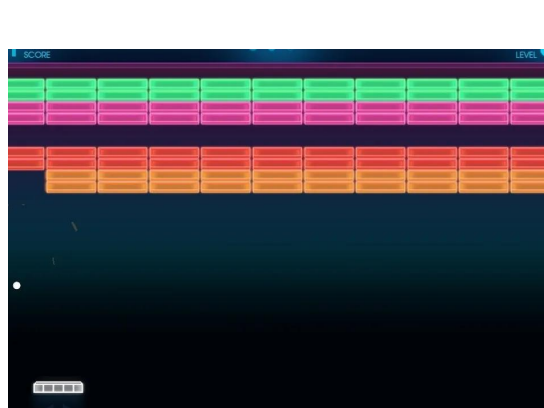
基于Actor-Critic的强化学习算法

- 结合两者优势

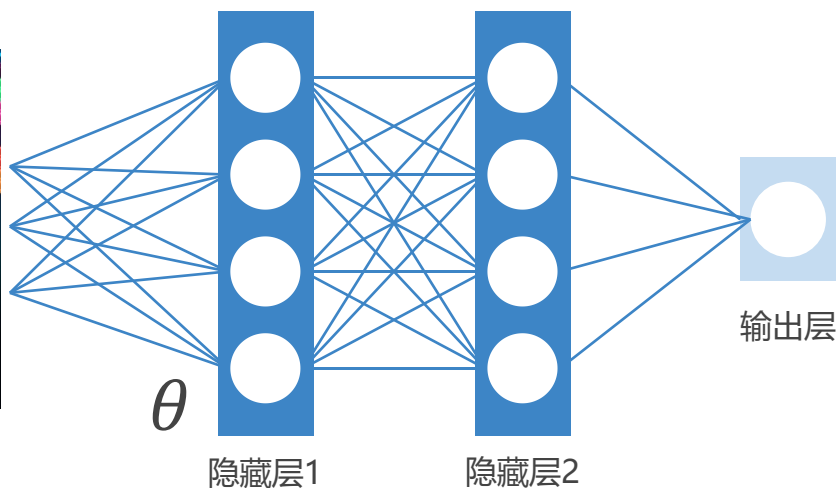
基于AC的RL

策略优化

- 基于策略的强化学习方法方法[直接搜索最优策略](#) π^*
- 通常做法是参数化策略 π_θ ，并利用无梯度或基于梯度的优化方法对参数进行更新
 - ✓ 无梯度优化可以有效覆盖低维参数空间，但基于梯度的训练仍然是首选，因为其具有更高的采样效率



输入状态 s



选择某一动作的概率

$$\pi_\theta(a|s)$$

基于AC的RL

策略表征方式不同也会带来不同的性质，下面给出参数化策略和表格型策略的三点不同之处

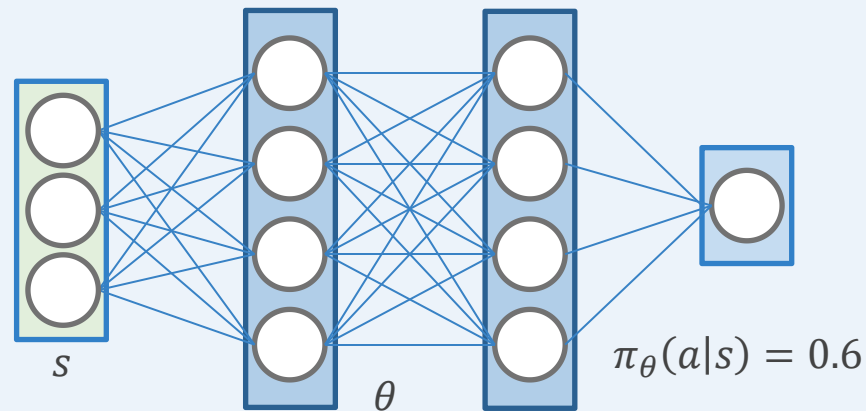
采取某个动作的概率的计算方式不同

- 表格型策略：状态 s 上采取动作 a 的概率直接查表可得
- 参数化策略：通过给定的函数结构和参数计算 $\pi_{\theta}(a|s)$

策略的更新方式不同

- 表格型策略：直接修改表格中对应的条目
- 参数化策略：通过更新参数 θ 对策略进行更新

	a_1	a_2
s_1	0.7	0.3
s_2	0.4	0.6



基于AC的RL

基于策略的强化学习算法

基本思想

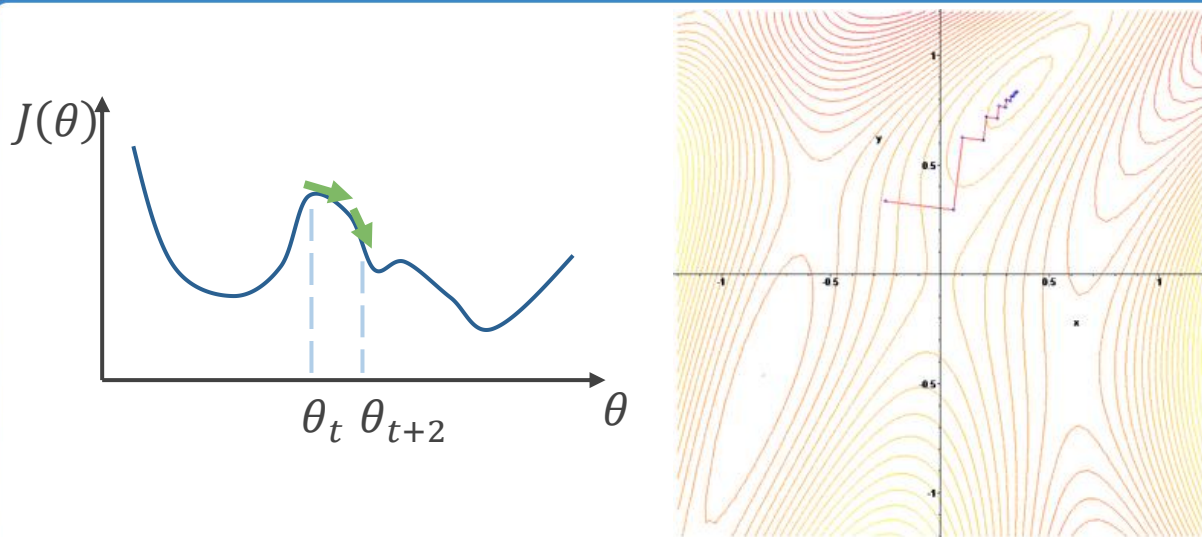
- 利用目标函数定义策略优劣性: $J(\theta) = J(\pi_\theta)$
- 对目标函数进行优化, 以寻找最优策略

两个问题

- 目标函数 $J(\theta)$ 如何设计?
- 该目标函数关于参数的优化方向 (如梯度 $\nabla_\theta J(\theta)$) 如何计算?

重点学习目标函数可微分情形

- 目标函数不可微分时: 使用无梯度算法进行最优参数搜索
- 目标函数可微分时: 利用基于梯度的优化方法寻找最优策略 $\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_\theta J(\theta_t)$



基于AC的RL

基于策略的强化学习算法——梯度优化

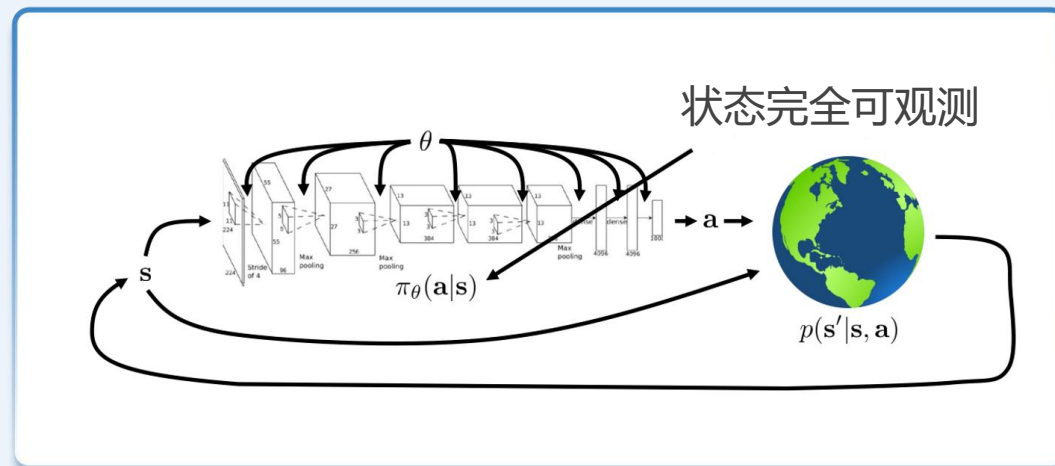
策略梯度计算

- 当目标函数可微分时，可以使用一些基于梯度的算法：
 - ✓ 梯度下降法、拟牛顿法等
- 但在此之前，需要先计算出目标函数关于策略参数的梯度。

最大化平均轨迹回报目标

$$\max_{\theta} J(\theta) = \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

- τ 为策略 π_{θ} 采样而来的轨迹
 $\{s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T\}$



$$\theta^* = \underset{\theta}{\operatorname{argmax}} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right]}_{\text{目标函数 } J(\theta)}$$

$$p_{\theta}(s_1, a_1, \dots, s_T) = p(s_1) \prod_{t=1}^{T-1} \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

基于AC的RL

基于策略的强化学习算法——梯度优化

策略梯度计算

- 记 $G(\tau) = \sum_t r(s_t, a_t)$, 轨迹回报目标的策略梯度为:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(\tau) \right]\end{aligned}$$

关于策略参数梯度的推导:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \int \nabla_{\theta} p_{\theta}(\tau) G(\tau) d\tau \\ \nabla_{\theta} J(\theta) &= \int \frac{p_{\theta}(\tau)}{p_{\theta}(\tau)} \nabla_{\theta} p_{\theta}(\tau) G(\tau) d\tau \\ \nabla_{\theta} J(\theta) &= \int p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} G(\tau) d\tau \\ \nabla_{\theta} J(\theta) &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau \\ \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) G(\tau)]\end{aligned}$$

基于AC的RL

基于策略的强化学习算法——梯度优化

策略梯度计算

关于 $\nabla_{\theta} \log p_{\theta}(\tau)$ 的简化:

$$\begin{aligned}\nabla_{\theta} \log p_{\theta}(\tau) &= \nabla_{\theta} (\log p(s_1) + \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) + \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t)) \\ &= \nabla_{\theta} \log p(s_1) + \nabla_{\theta} \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) + \nabla_{\theta} \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t) \\ &= \nabla_{\theta} \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) \\ &= \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

其中 $p(s_1)$, $p(s_{t+1} | s_t, a_t)$ 与参数 θ 无关, 因此 $\nabla_{\theta} \log p(s_1) = 0$, $\nabla_{\theta} \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t) = 0$

基于AC的RL

REINFORCE 算法

基于蒙特卡洛方法计算策略梯度

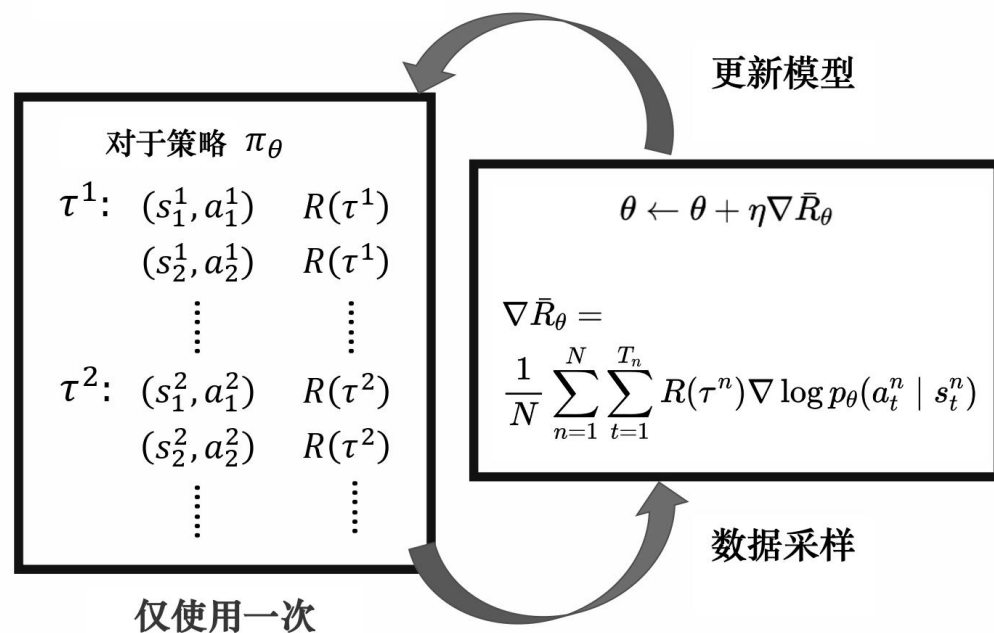
- 前面我们推导出了策略梯度:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

- 在实践中, 我们可以通过蒙特卡洛方法进行估计

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T^n} R(\tau^n) \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n)$$

- 据此, 我们可以得到 REINFORCE 算法



REINFORCE 算法:

1. 利用策略 $\pi_{\theta}(a|s)$ 采样轨迹 $\{\tau_i\}$
2. 计算梯度 $\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T^n} R(\tau^n) \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n)$
3. 更新参数 $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

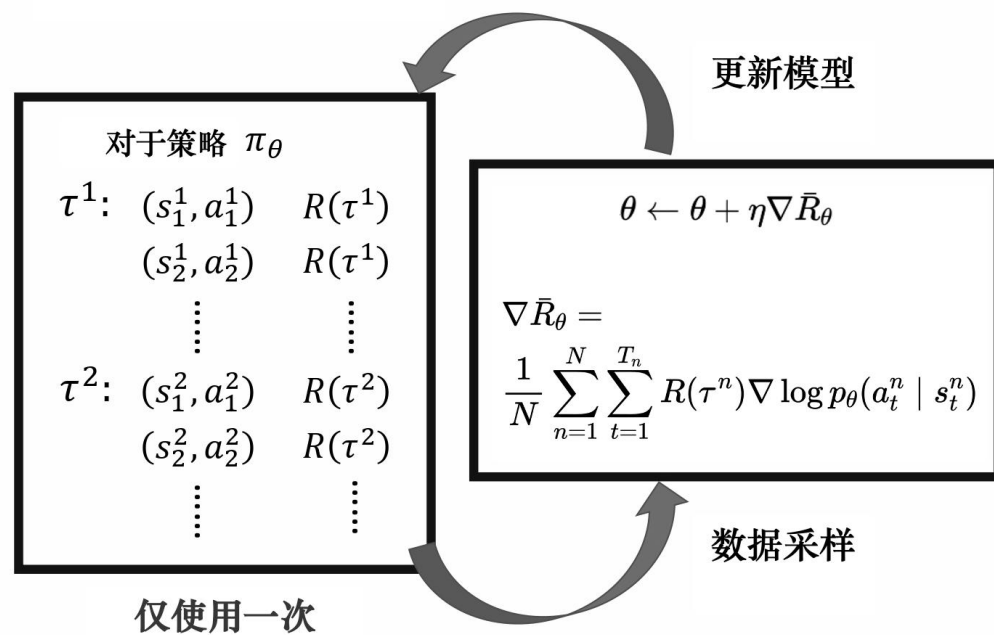
基于AC的RL

REINFORCE 算法

同策略 (On-Policy) 算法

$$\begin{aligned}\theta^* &= \operatorname{argmax}_{\theta} J(\theta) \\ J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)] \\ \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]\end{aligned}$$

估计策略梯度所用的数据需要由当前策略采集



REINFORCE 算法:

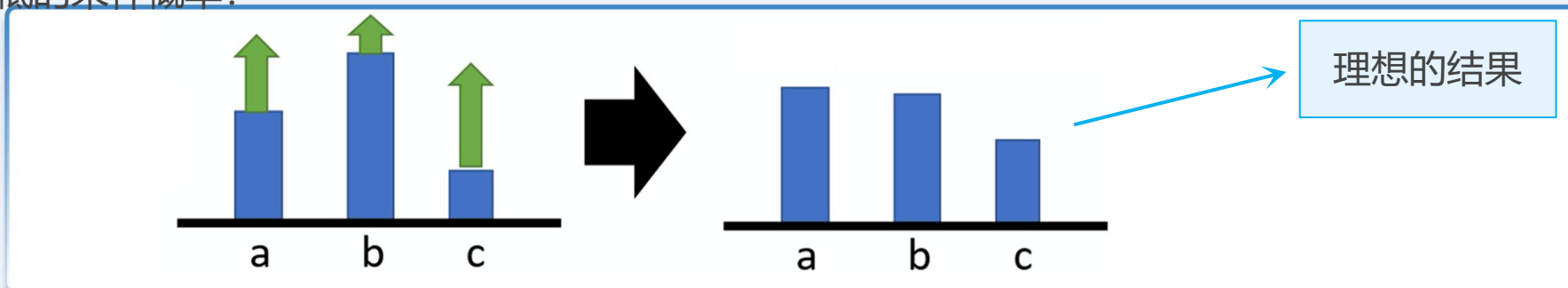
1. 利用策略 $\pi_{\theta}(a|s)$ 采样轨迹 $\{\tau_i\}$
2. 计算梯度 $\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n)$
3. 更新参数 $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

基于AC的RL

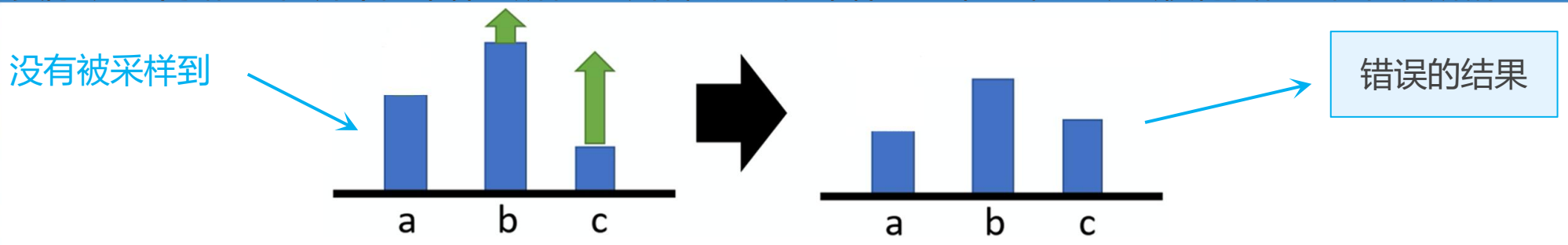
REINFORCE 算法

训练可能存在偏差

- 考虑只有正奖励的场景，在理想情况下进行优化，最终能够使得奖励值高的动作分配较高的采样概率，而奖励值低的动作分配较低的采样概率：



- 但在实际场景中我们可能并不能采样到所有的动作，如只能采样到 b 和 c，这就会使得我们的优化有所偏差：



基于AC的RL

REINFORCE 算法

通过添加基线帮助训练

$$\nabla R_{\theta} = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T^n} (R(\tau^n) - b) \nabla_{\theta} \log p_{\theta}(a_t^n | s_t^n)$$

- 我们可以将奖励函数减去一个基线 b ，使得 $R(\tau) - b$ 有正有负
 - ✓ 如果 $R(\tau) > b$ ，就让采取对应动作的概率提升
 - ✓ 如果 $R(\tau) < b$ ，就让采取对应动作的概率降低
- 一般来说，可以使用奖励的平均值作为基线， $b = \frac{1}{N} \sum_{i=1}^N R(\tau)$ 。
 - ✓ 在训练时记录 $R(\tau)$ 的值，并维护 $R(\tau)$ 的平均值

基于AC的RL

REINFORCE 算法

通过添加基线帮助训练

- 减去一个基线并不会影响原梯度的期望值

$$\begin{aligned}\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau) b] &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b \, d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) b \, d\tau \\ &= b \int \nabla_{\theta} p_{\theta}(\tau) \, d\tau \\ &= b \nabla_{\theta} \int p_{\theta}(\tau) \, d\tau \\ &= b \nabla_{\theta} 1 = 0\end{aligned}$$

基于AC的RL

基于Actor-Critic的强化学习算法

REINFORCE 算法主循环

1. 使用当前策略 π_θ 采样N条经验轨迹 $\{\tau_i\}$

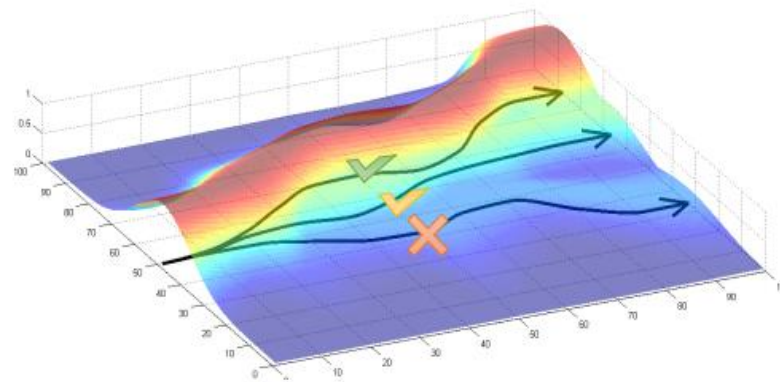
对第t步做对数极大似然估计

2. 计算梯度 $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right) \right)$

对N条轨迹进行平均

在每一条轨迹内累计回报

3. 更新策略参数 $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



基于AC的RL

基于Actor-Critic的强化学习算法

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \underbrace{\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)}_{\text{方差较大}} \right)$$

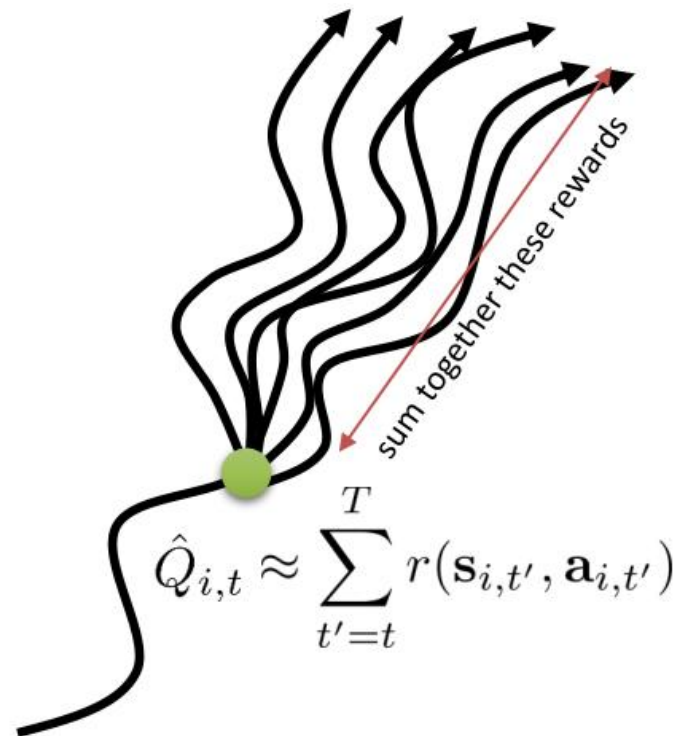
改进

方差较大

使用动作价值估计 $\hat{Q}^{\pi}$ 近似轨迹回报期望 $\mathbb{E}_{\pi_{\theta}} \sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \underbrace{\hat{Q}^{\pi}(s_t^i, a_t^i)}_{\text{改进}} \right)$$

$$\hat{Q}^{\pi}(s_t^i, a_t^i) \approx \sum_{t'=t}^T \mathbb{E}_{\pi_{\theta}}[r(s_{t'}, a_{t'}) | s_t, a_t]$$



基于AC的RL

基于Actor-Critic的强化学习算法

• Actor: π_θ • Critic: $\hat{Q}^\pi$ 1. 使用当前策略 π_θ 在环境中进行采样2. 策略提升: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \hat{Q}^\pi(s_t^i, a_t^i) \right)$ 3. 拟合当前策略的动作值函数: $\hat{Q}^\pi(s_{t'}^i, a_{t'}^i) \approx \sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$ 对第 t 步做对数极大似然估计对 N 条轨迹进行平均

动作值函数估计(Critic)

$$\hat{Q}^\pi(s_{t'}^i, a_{t'}^i) = \sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$$

fit a model to estimate return

generate samples (i.e. run the policy)

improve the policy

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Actor Learning

Critic Learning

基于AC的RL

基于Actor-Critic的强化学习算法

Actor-Critic算法的策略梯度

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{Q}^{\pi}(s_t^i, a_t^i) \right) \quad (*)$$

进一步对策略梯度进行改进

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \underbrace{\hat{A}^{\pi}(s_t^i, a_t^i)}_{\text{减去基线函数} \hat{V}^{\pi}(s_t^i)} \right) \quad (**)$$

“优势函数” : $\hat{A}^{\pi}(s_t^i, a_t^i) = \hat{Q}^{\pi}(s_t^i, a_t^i) - \hat{V}^{\pi}(s_t^i)$

Advantage Actor-Critic (A2C) 算法

减去基线函数 $\hat{V}^{\pi}(s_t^i)$

基于AC的RL

基于Actor-Critic的强化学习算法

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{A}^{\pi}(s_t^i, a_t^i) \right) \quad (**)$$

Remark: 关于优势函数 $\hat{A}^{\pi}(s_t^i, a_t^i)$:

- 含义：衡量当前动作 a_t^i 相对于平均情况 $\hat{V}^{\pi}(s_t^i)$ 的好坏程度
- 等价性：用 $\hat{A}^{\pi}$ 代替 $\hat{Q}^{\pi}$ 后策略梯度期望值不变，证明：
- 作用：对单个动作的梯度有正负之分，减小方差

$$\nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{Q}^{\pi}(s_t^i, a_t^i) = \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) (\hat{Q}^{\pi}(s_t^i, a_t^i) - \hat{V}^{\pi}(s_t^i)) + \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{V}^{\pi}(s_t^i)$$

$$\text{而 } \mathbb{E}_{a_t \sim \pi_{\theta}(a_t | s_t)} [\nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{V}^{\pi}(s_t^i)] =$$

$$\sum_{a_t^i \in \mathcal{A}} \pi_{\theta}(a_t^i | s_t^i) \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{V}^{\pi}(s_t^i) =$$

$$\sum_{a_t^i \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a_t^i | s_t^i) \hat{V}^{\pi}(s_t^i) =$$

$$\hat{V}^{\pi}(s_t^i) \nabla_{\theta} \sum_{a_t^i \in \mathcal{A}} \pi_{\theta}(a_t^i | s_t^i) = \hat{V}^{\pi}(s_t^i) \nabla_{\theta} 1 = 0,$$

可知使用 $\hat{A}^{\pi}(s_t^i, a_t^i) = \hat{Q}^{\pi}(s_t^i, a_t^i) - \hat{V}^{\pi}(s_t^i)$ 代替 $\hat{Q}^{\pi}(s_t^i, a_t^i)$ 不会影响策略梯度的期望值。

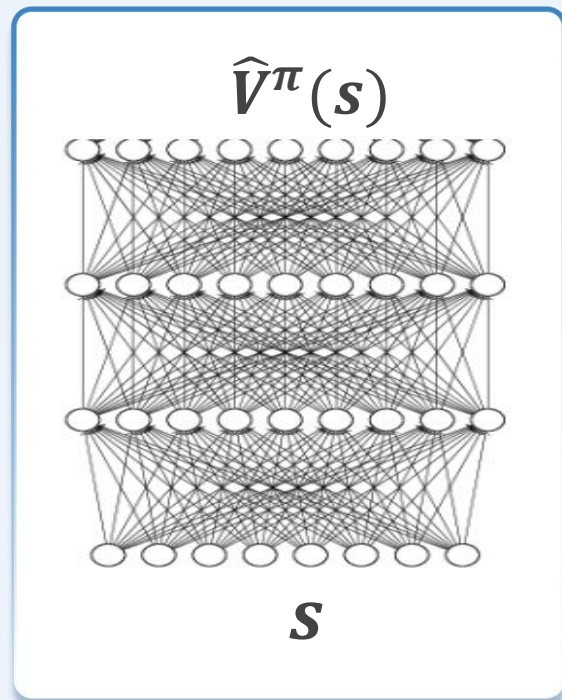
基于AC的RL

基于Actor-Critic的强化学习算法

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \hat{A}^{\pi}(s_t^i, a_t^i) \right) \quad (**)$$

$$\begin{aligned} \nabla_{\theta} J(\theta) &\approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) [\hat{Q}^{\pi}(s_t^i, a_t^i) - \hat{V}^{\pi}(s_t^i)] \right) \\ &= \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) [R(s_t^i, a_t^i) + \gamma \hat{V}^{\pi}(s_{t+1}^i) - \hat{V}^{\pi}(s_t^i)] \right) \end{aligned}$$

只需用一个神经网络拟合 $\hat{V}^{\pi}$!



基于AC的RL

基于Actor-Critic的强化学习算法

- 基于AC算法的一般表达式是

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \Psi_t \right)$$

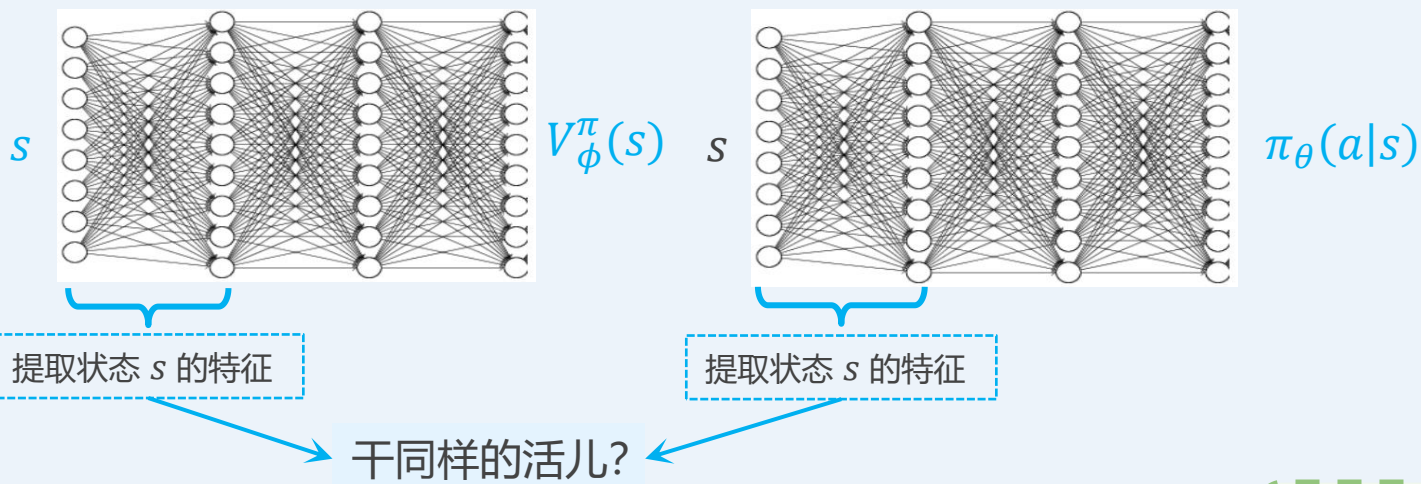
其中 Ψ_t 是值函数的某种表达式。我们列举一些可能的 Ψ_t

1. $\Psi_t = V_t$, 用状态值函数更新策略梯度
2. $\Psi_t = Q_t$, 用状态动作值函数更新策略梯度
3. $\Psi_t = A_t$, 用动作优势函数更新策略梯度, 即A2C算法
4. $\Psi_t = TD_t$, 用TD Error更新策略梯度
5. $\Psi_t = G_t - V_t$, 用状态值函数作为基线更新策略梯度

基于AC的RL

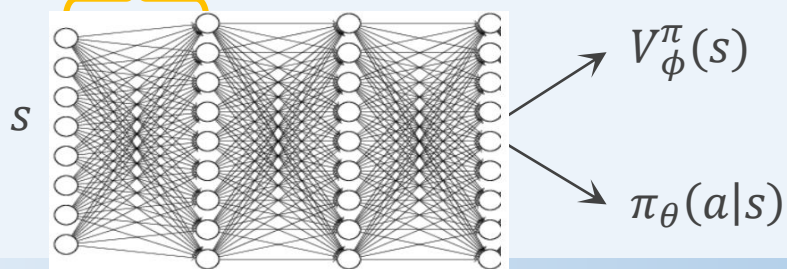
有两种神经网络架构方案

方案一：实现两个神经网络分别对价值函数 $\hat{V}^\pi$ 和策略 π_θ 进行拟合：



- 优点：简单且训练稳定
- 缺点：训练效率较低

方案二：共享前几层特征提取网络，只使用一个神经网络：



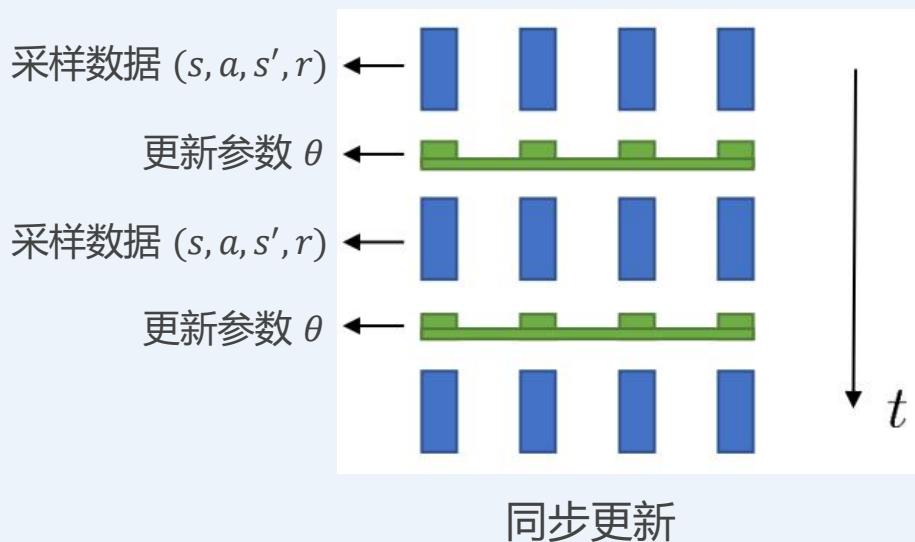
- 优点：减小了参数量，因此训练效率高
- 缺点：由于策略与值所需的状态特征有一定差异，因此训练稳定性上要差于方案一

基于AC的RL

基于Actor-Critic的强化学习算法

利用单个样本进行更新：更新方差较大，训练稳定性差

解决方案：获得一个批次的数据后再进行更新



Algorithm 1.12 AC 算法

Input: 迭代轮数 T , 学习率 α , 衰减因子 γ , 最大交互回合数 N , 最大时间步长 T , 存储队列 $\mathcal{B}$

Output: 最优策略 π_*

- 1: 随机初始化策略网络 π_θ , 值网络 ϕ
- 2: **for** 每个交互回合 $k = 0, 1, \dots, N$ **do**
- 3: 重置环境并获取环境的初始状态 s_0
- 4: **for** $t = 0, 1, \dots, T$ **do**
- 5: 在状态 s_t , 通过策略网络 π_θ 选择动作 a_t
- 6: 在状态 s_t 执行当前动作 a_t , 得到新状态 s_{t+1} 和奖励 r_t
- 7: 计算 $\log(\pi_\theta(a_t|s_t))$
- 8: 将 $\log(\pi_\theta(a_t|s_t))$ 、 s_t 、 a_t 、 r_t 、 s_{t+1} 存入存储队列 $\mathcal{B}$
- 9: **end for**
- 10: 从存储队列 $\mathcal{B}$ 取出交互回合 k 的轨迹数据
- 11: 计算每个状态的 Ψ_t
- 12: 利用策略更新公式和值函数更新公式分别更新 θ 和 ϕ
- 13: 清空存储队列 $\mathcal{B}$
- 14: **end for**

在Gym环境下DQN的实现代码

```
def main():  
    env = gym.make(args.env)  # 创建环境对象env  
    o_dim = env.observation_space.shape[0]  # 从环境对象中获取观测维度和动作维度  
    a_dim = env.action_space.n  
    agent = DQN(env, o_dim, args.hidden, a_dim)  # 创建智能体对象agent  
    for i_episode in range(args.n_episodes):  
        obs = env.reset()  
        episode_reward = 0  
        done = False  
        while not done:  
            action = agent.choose_action(obs)  
            next_obs, reward, done, info = env.step(action)  
            agent.store_transition(obs, action, reward, next_obs, done)  # 交互数据存入buffer  
            episode_reward += reward  
            obs = next_obs  
        if agent.buffer.len() >= args.capacity:  
            agent.learn()  # DQN网络训练
```

智能体和环境交互

在Gym环境下DQN的实现代码

```
class DQN:
    def __init__(self, env, input_size, hidden_size, output_size):
        self.env = env
        self.eval_net = QNet(input_size, hidden_size, output_size)
        self.target_net = QNet(input_size, hidden_size, output_size)
        self.optim = optim.Adam(self.eval_net.parameters(), lr=args.lr)
        self.eps = args.eps
        self.buffer = ReplayBuffer(args.capacity)
        self.loss_fn = nn.MSELoss()
        self.learn_step = 0

    def choose_action(self, obs):
        if np.random.uniform() <= self.eps:
            action = np.random.randint(0, self.env.action_space.n)
        else:
            action_value = self.eval_net(obs)
            action = torch.max(action_value, dim=-1)[1].numpy()
        return int(action)

    def store_transition(self, *transition):
        self.buffer.push(*transition)
```

创建神经网络

创建buffer

利用 ϵ -greedy选择动作

在Gym环境下DQN的实现代码

```
def learn(self):  
    if self.eps > args.eps_min:  
        self.eps *= args.eps_decay
```

→ ϵ 随着学习的次数增加逐渐下降

```
    if self.learn_step % args.update_target == 0:  
        self.target_net.load_state_dict(self.eval_net.state_dict())  
    self.learn_step += 1
```

→ 目标网络间隔更新

从buffer中采样一个
mini-batch 数据

```
    obs, actions, rewards, next_obs, dones = self.buffer.sample(args.batch_size)  
    actions = torch.LongTensor(actions) # LongTensor to use gather latter  
    dones = torch.IntTensor(dones)  
    rewards = torch.FloatTensor(rewards)
```

计算TD Error, 更
新Q网络

```
    q_eval = self.eval_net(obs).gather(-1, actions.unsqueeze(-1)).squeeze(-1)  
    q_next = self.target_net(next_obs).detach()  
    q_target = rewards + args.gamma * (1 - dones) * torch.max(q_next, dim=-1)[0]  
    loss = self.loss_fn(q_eval, q_target)  
    self.optim.zero_grad()  
    loss.backward()  
    self.optim.step()
```




在Gym环境下DQN的实现代码

```
class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = []
        self.capacity = capacity

    def len(self):
        return len(self.buffer)

    def push(self, *transition):
        if len(self.buffer) == self.capacity:
            self.buffer.pop(0)
        self.buffer.append(transition)

    def sample(self, n):
        index = np.random.choice(len(self.buffer), n)
        batch = [self.buffer[i] for i in index]
        return zip(*batch)

    def clean(self):
        self.buffer.clear()
```

```
class QNet(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(QNet, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = torch.Tensor(x)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

多智能体强化学习

单智能体强化学习方法 (Single-Agent Reinforcement Learning, SARL) :

- 只有一个智能体与环境交互，环境的状态转移和奖励完全由环境本身决定；
- 智能体的决策是独立的；
- 环境是“静态”的，智能体在学习过程中假设环境不会改变；
- 奖励完全由环境反馈给智能体，奖励函数通常是单一的且只考虑智能体的行为对环境的影响。

多智能体强化学习 (Multi-Agent Reinforcement Learning, MARL) :

- 有多个智能体与环境同时交互；
- 智能体的决策是相互依赖的，每个智能体的动作不仅影响环境状态，还会影响其他智能体的状态和行为；
- 环境是“非平稳”的。因为每个智能体的行为都会影响到其他智能体的决策，智能体在学习过程中无法假设其他智能体的策略是固定的；
- 每个智能体可能有自己的奖励函数。在合作场景中，智能体的奖励函数可能是共享的；在竞争场景中，每个智能体的奖励通常是独立的。

多智能体强化学习

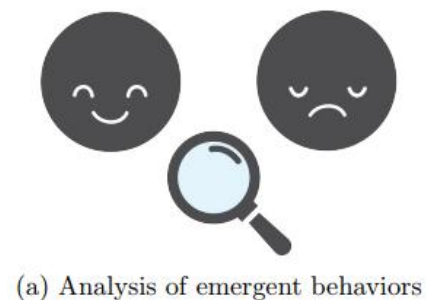
多智能体强化学习所面临的困境：

- **环境的非平稳性 (Non-stationarity)**：在多智能体环境中，由于每个智能体的行为都会影响环境状态，并且可能改变其他智能体的策略，因此整个系统是非平稳的。
- **部分可观测性 (Partial Observability)**：智能体无法完全观察到全局的环境状态，而只能获得自己局部的观察信息，限制了其决策能力。
- **维度灾难 (Curse of dimensionality)**：随着智能体的数量的增长，所有智能体的联合状态和联合动作空间呈指数级增长。
- **策略协同与通信 (Strategy Coordination and Communication)**：在合作任务中，智能体需要通过协同来共同优化整个团队的目标。在某些场景中，智能体可以通过直接的通信来交换信息或传递意图。
- **博弈与对抗 (Game Theory and Adversarial Behavior)**：在竞争性环境中，多个智能体之间的行为往往是对抗性的（如零和博弈），每个智能体都试图最大化自己的奖励，而不是共享奖励。
- ...

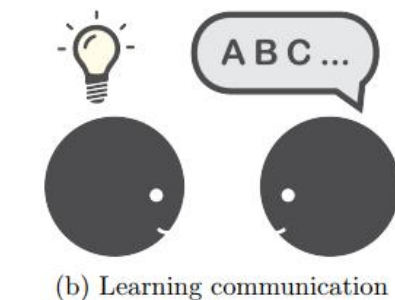
多智能体强化学习

多智能体强化学习算法分类：

- **独立学习（或者称行为分析）**：在多智能体场景下，使用单智能体算法独立控制每个智能体。
- **通信机制**：智能体与智能体在交互过程中可以传递信息。
- **协同学习**：依据局部观测，利用一些协同机制实现智能体之间的协同。
- **智能体建模**：推理其他智能体的策略模型。



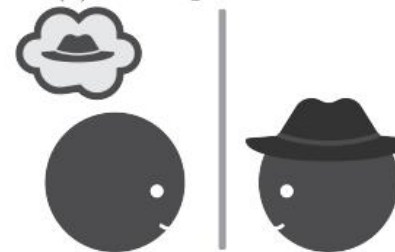
(a) Analysis of emergent behaviors



(b) Learning communication



(c) Learning cooperation



(d) Agents modeling agents

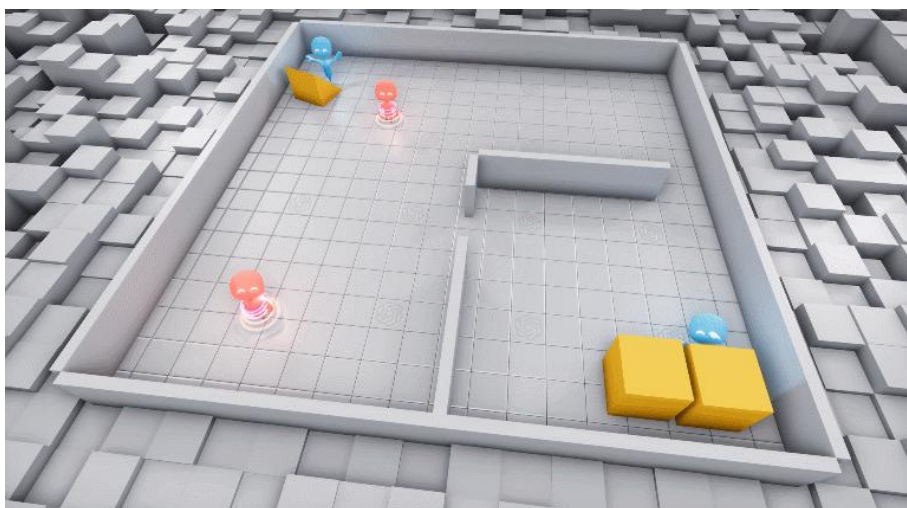
多智能体强化学习

重点介绍“合作型”的 MARL 算法

多智能体场景分类：

- **合作型场景：**所有智能体合作完成某一任务，学习目标是最大化团队奖励。
- **竞争性场景：**智能体之间争夺有限的资源或追求个体利益，学习目标是最大化个体奖励。
- **混合场景：**部分智能体是协同关系，但与其他智能体是竞争关系。

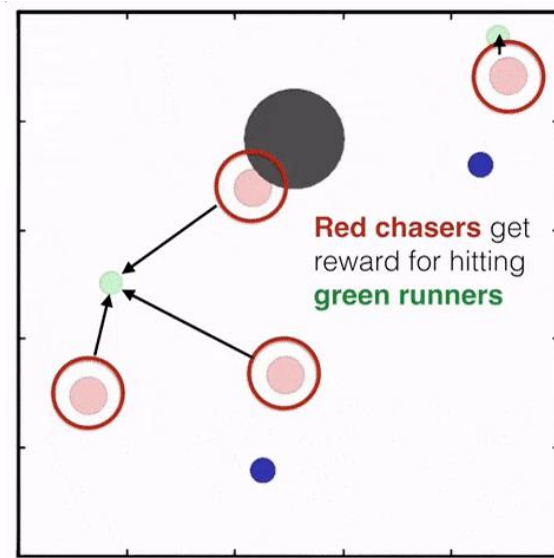
合作型场景



竞争性场景



混合场景

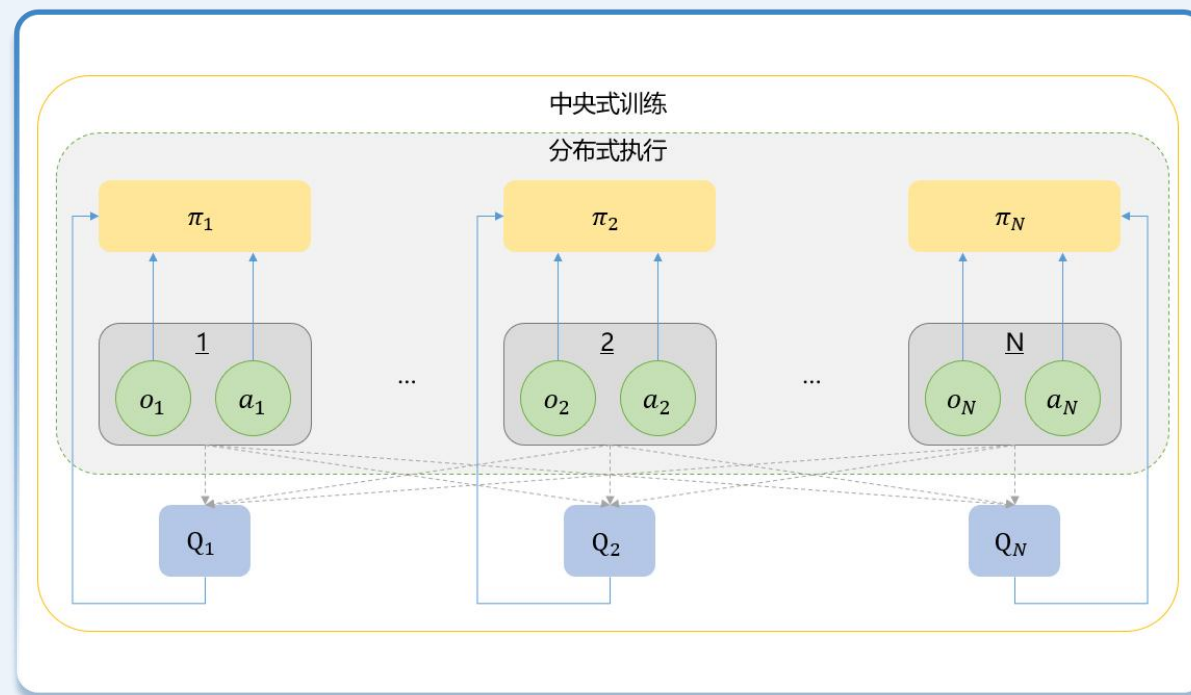


MARL算法

基于策略的多智能体强化学习——MADDPG

MADDPG (Multi-Agent Deep Deterministic Policy Gradient) 利用**中央式训练分布式执行 (Centralized Training with Decentralized Execution, CTDE)** 框架解决多智能体环境中的环境不稳定性以及智能体协同问题:

- 在训练阶段, MADDPG算法利用所有智能体的状态和动作信息进行值函数的全局优化。通过引入中心化的值函数, MADDPG能够有效地捕获智能体间的相互影响, 缓解了环境的不稳定性问题。
- 在执行过程中, 每个智能体基于自己的局部观测来进行决策。即使智能体只能访问到自己的局部信息, 依据在训练阶段获得的全局信息, 智能体依然能够采取协作策略。



MARL算法

基于值函数的多智能体强化学习——VDN

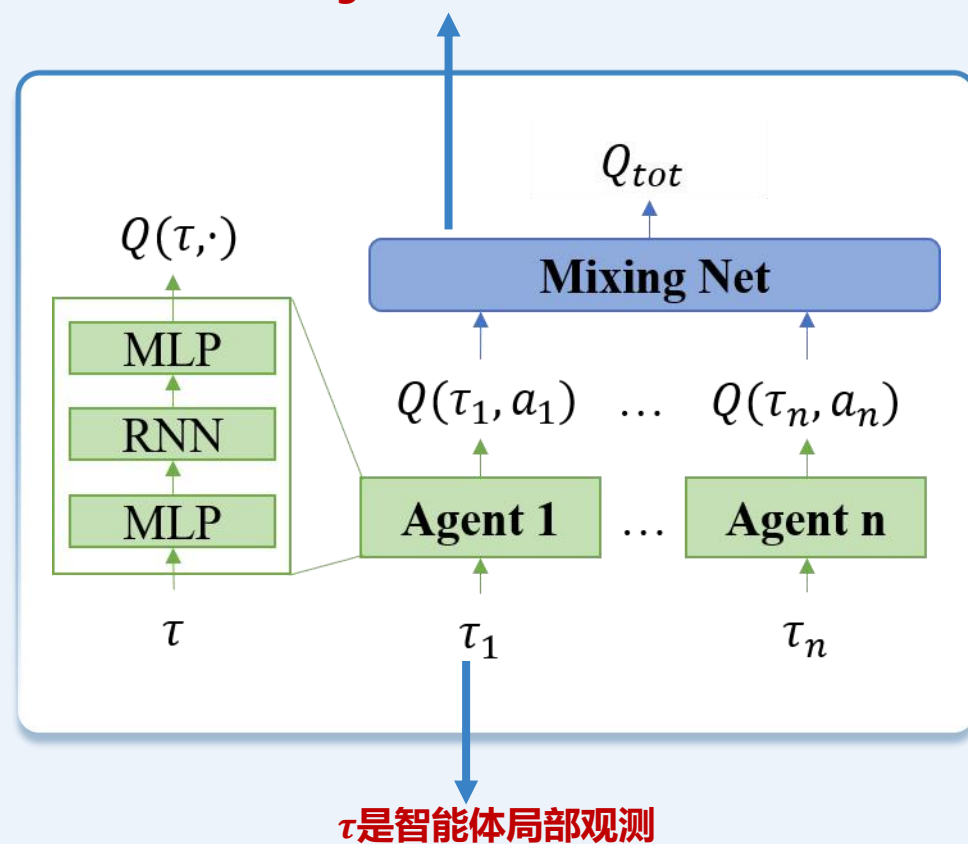
MADDPG算法存在两个问题：

- **值函数学习困难：**值函数所拟合的联合状态空间的维度随着智能体数量呈指数级增加。
- **信任分配问题：**共享团队奖励导致智能体难以区分自身对团队的贡献，导致智能体无法有效地学习如何根据自己的局部信息做出有贡献的动作。

VDN (Value Decomposition Networks) 算法通过值分解的方式解决上述两问题。

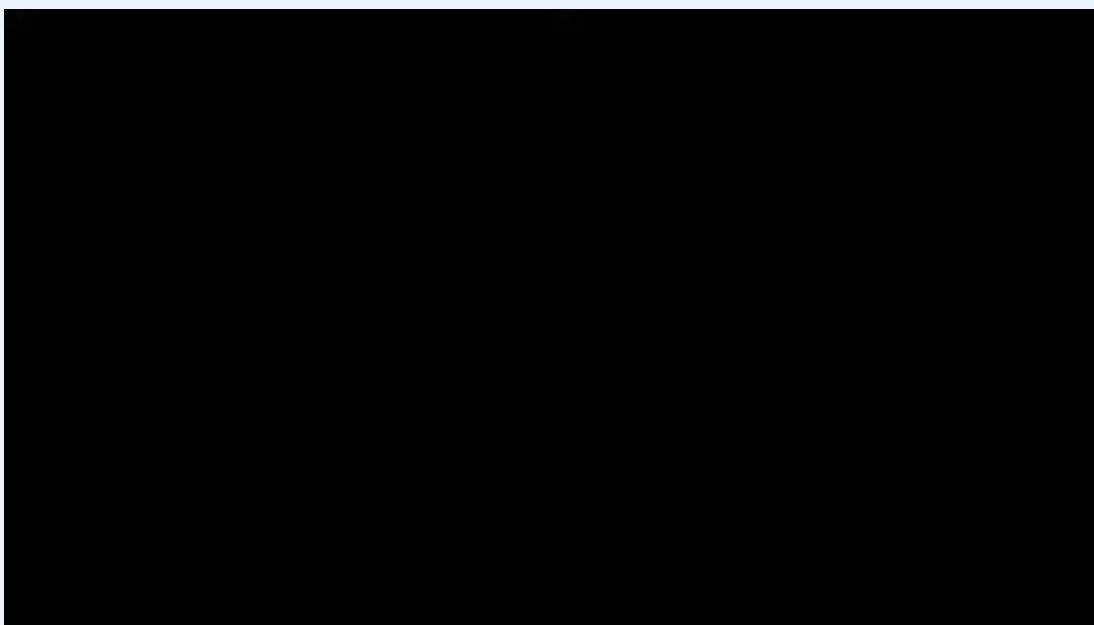
- 全局的Q值函数是由所有智能体的局部Q值累加得到。
- 通过将全局 Q 函数分解为局部 Q 函数之和，VDN隐含地表示了每个智能体对团队目标的相对贡献，即每个智能体自身的Q值表示对团队的共享。
- VDN仍是CTDE框架，在执行过程中，每个智能体依据自身的局部值函数进行决策，在训练过程中，利用全局值函数进行学习

Mixing Net 是一个线性求和函数

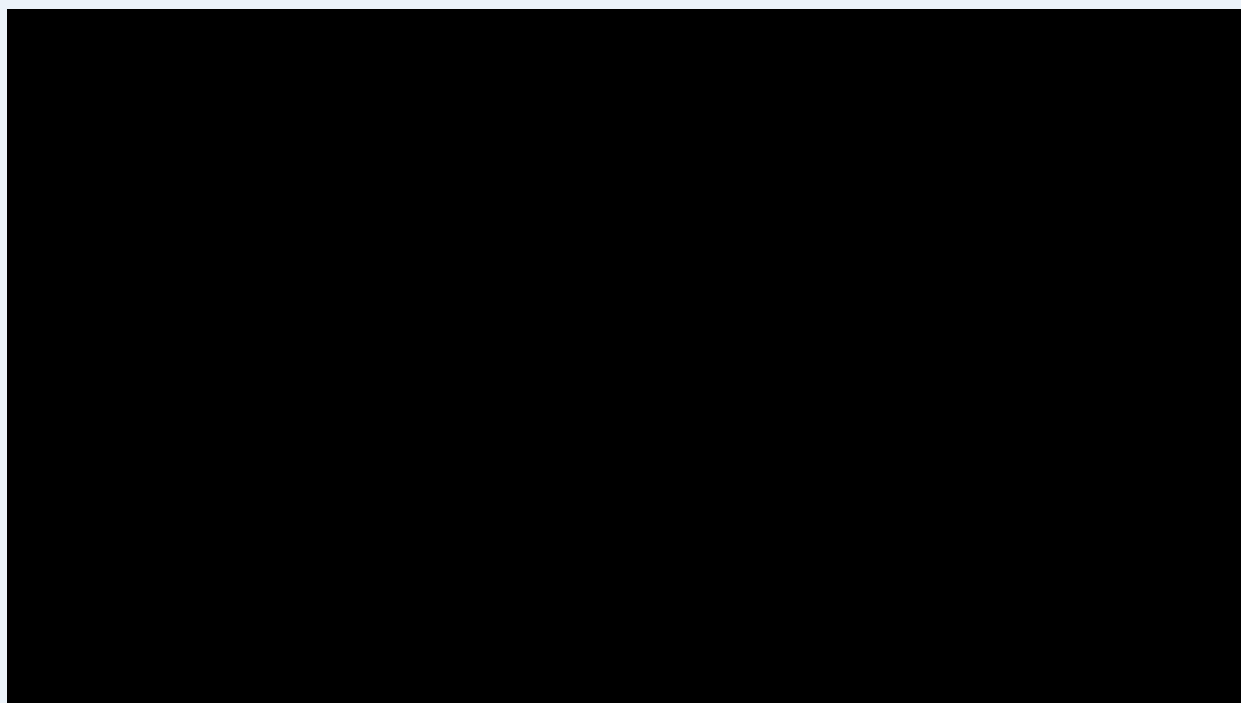




应用——机器人



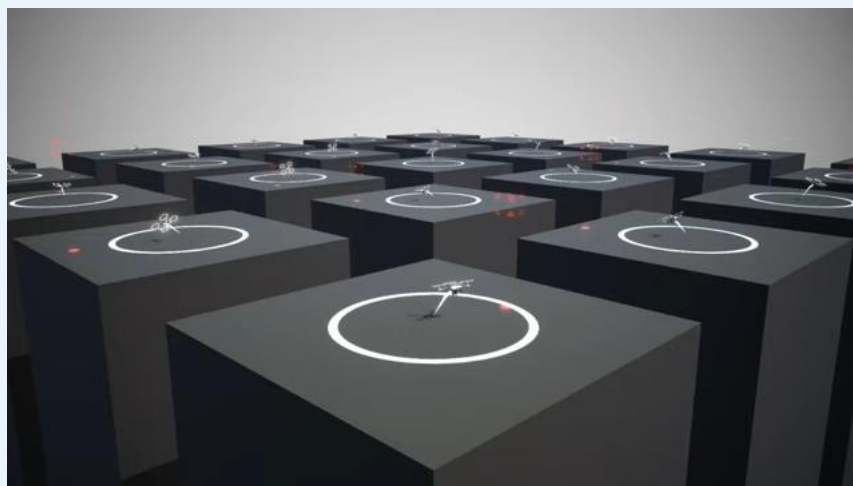
应用——游戏



应用——无人机集群

Multi-agent Reinforcement Learning Formation Control

Purdue AIMS Lab
Tianyu Zhou, Shaoshuai Mou

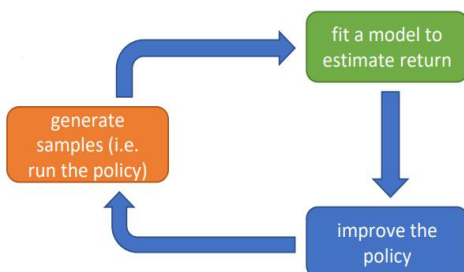


本节小结

单智能体RL

- 基于值的RL: DQN及其改进: Double DQN、Dueling DQN、Rainbow DQN
- 基于策略的RL: REINFORCE算法
- 基于AC的RL: AC、A2C

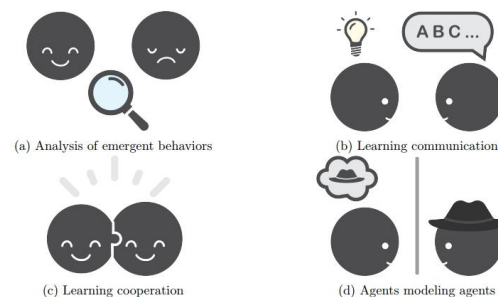
$$\hat{Q}^{\pi}(s_{t'}^i, a_{t'}^i) = \sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$$

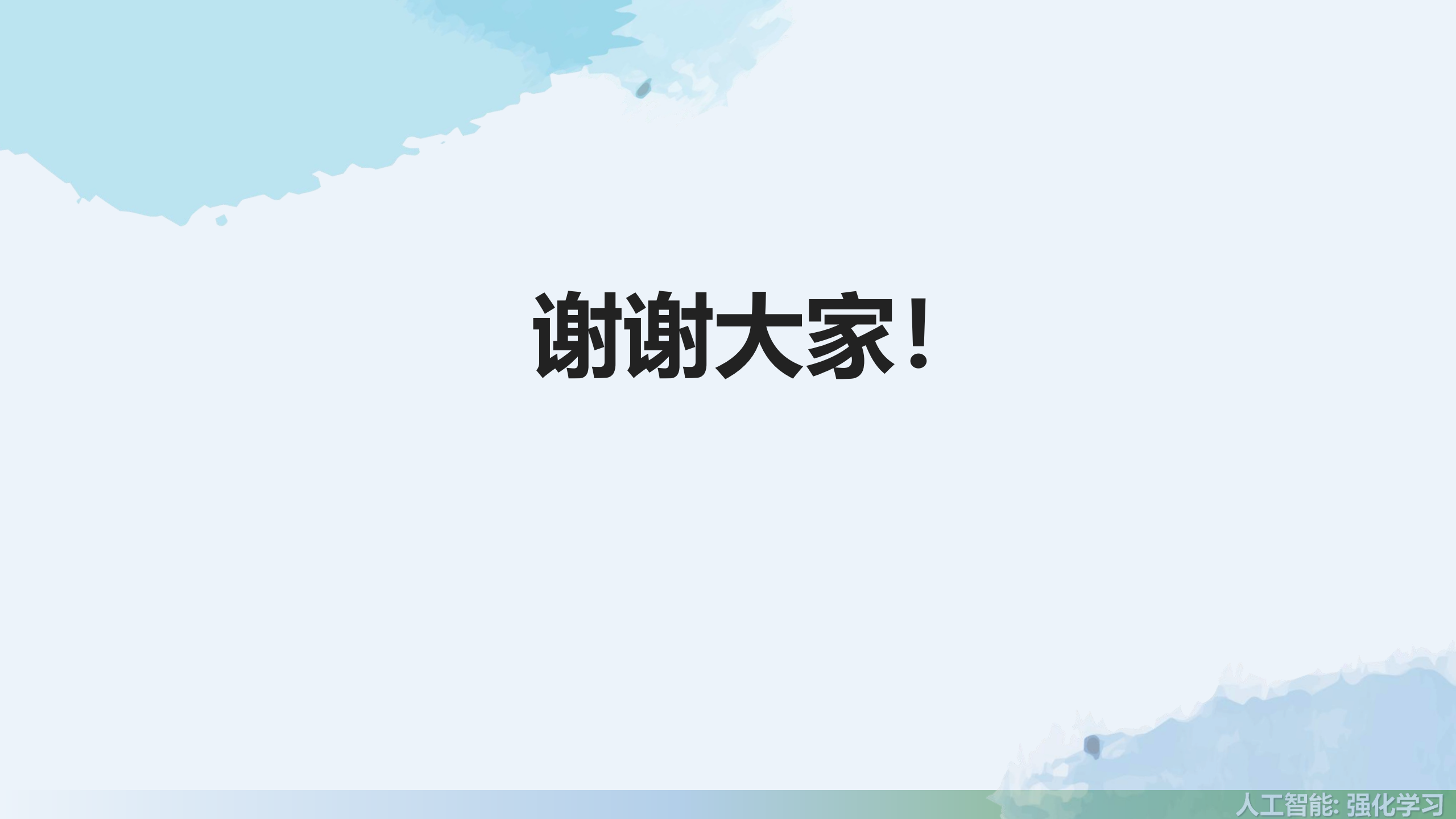


$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

多智能体RL

- 多智能体定义
- 多智能体RL困境
- 多智能体RL算法分类
- 多智能体场景分类
- 多智能体RL应用





谢谢大家!